# 2026-2027 9. Sınıf Programlamanın Temelleri

## İçindekiler


1. Öğrenme Birimi Algoritmik Düşünme ve Tasarım ..... 1
   1.1. Problem Analizi ve Bileşenleri ..... 1
      1.1.1. Algoritma ve Tasarım Kriterleri ..... 1
      1.1.2. Problemi Bileşenlerine Ayırma ..... 1
      1.1.3. Girdi-İşlem-Çıktı Modeli ..... 1
   1.2. Akış Şeması (Flowchart) ..... 2
      1.2.1. Sözde Kod ve Elle Çalıştırma ..... 2
         1.2.1.1. Örnek 1: İki Sayının Toplamını Bulma (Sayfa 25 Örnek 1) ..... 2
         1.2.1.2. Karşılaştırma Operatörleri ve Kullanım Örnekleri ..... 3
            1.2.1.2.1. Karşılaştırma Operatörleri İle Yapılan İşlem Örnekleri ..... 3
         1.2.1.3. Örnek 1 Hatalı Giriş Kontrolü Algoritması (Sayfa 28 Örnek 4) ..... 4
         1.2.1.4. Örnek 2 Sıcaklık Kontrolü (Sayfa 28 Örnek 3) ..... 5
         1.2.1.5. Örnek 3: Yaş Kontrolü ve Ehliyet Alabilme (Sayfa 25 Örnek 2) ..... 6
         1.2.1.6. Flowgorithm ve Thonny Uygulamaları ..... 7
         1.2.1.7. Değişkenler ve Veri Tipleri ..... 8
            1.2.1.7.1. Temel Veri Tipleri ..... 8
            1.2.1.7.2. Değişken İsimlendirme Kuralları ..... 8
      1.2.2. Akış Şeması (Flowchart) Sembolleri ve Görsel Modelleme ..... 9
         1.2.2.1. Örnek 1: İki Sayının Toplamını Bulma (Sayfa 25 Örnek 1) ..... 9
         1.2.2.2. Örnek 2: İki Sayının Ortalamasını Bulma ..... 9
         1.2.2.3. Örnek Çalışma Soruları ..... 10
         1.2.2.4. Karar Yapıları ..... 10
         1.2.2.5. Örnek 3 Sıcaklık Kontrolü ..... 11
         1.2.2.6. Örnek 4 Hatalı Giriş Kontrolü Algoritması ..... 12
         1.2.2.7. Örnek 5: Yaş Kontrolü ve Ehliyet Alabilme ..... 13
         1.2.2.8. Örnek Çalışma Soruları Karar Yapıları ..... 14
         1.2.2.9. Döngü(Loop) ..... 15
         1.2.2.10. Örnek 6 Döngüsel Ekran Yazımı (Sayfa 37 Örnek 8) ..... 15
         1.2.2.11. Örnek 7 Ardışık Sayıların Toplamı (Sayfa 38 Uygulama 4) ..... 16
         1.2.2.12. Örnek Çalışma Soruları Döngü Konusu ..... 18
      1.2.3. Algoritmalarda Metinsel Veriler ve İşlemler ..... 19
         1.2.3.1. Örnek Metin Birleştirme (Sayfa 42 Örnek 9) ..... 19
      1.2.4. Dijital Depo ..... 19
2. Temel Programlama Yapıları ..... 20
   2.1. Değişkenler, Sabitler ve Veri Türleri ..... 20
      2.1.1. Donanımın Temel Yapı Taşı: Transistör ve İkili Sistem (Binary) ..... 20
      2.1.2. İlk Bilgisayarlar ve 8 Bitlik Komut Girişleri (Altair 8800) ..... 20
      2.1.3. Sayı Sistemleri ve Gruplama: 4 Bit ve Onaltılık (Hexadecimal) ..... 21
      2.1.4. Metin Temsili: 8 Bit = 1 Byte ve ASCII Tablosu ..... 21
      2.1.5. Dijital Dünyanın Ölçü Birimi Bit'ten Terabayt'a Veri Büyüklükleri ..... 22
      2.1.6. Makine Kodundan Assembly Diline ..... 23
      2.1.7. Programlama Dilleri ..... 24
      2.1.8. Değişkenler ve Temel Veri Türleri ..... 27
         2.1.8.1. Değişken ..... 27
         2.1.8.2. Temel Veri Türleri ..... 28
         2.1.8.3. Değişken Tanımlama Kuralları ..... 28

2.1.8.4. Değişken Adlandırma Standartları (Notasyonlar) ..... 29
2.1.8.5. Sabitler ..... 29
2.1.8.6. Açıklama Satırı (Comment line) ..... 29
2.1.9. Python Dili ..... 30

# 2026-2027 9. Sınıf Programlamanın Temelleri

# 1. Öğrenme Birimi Algoritmik Düşünme ve Tasarım

## 1.1. Problem Analizi ve Bileşenleri

| ANALİZ |
| --- |
| Bir olayı veya durumu incelemek ve ayrıntılı biçimde anlamaya çalışmaktır. |

Programlama sürecinde problem analizi, çözülmesi gereken sorunu dikkatlice inceleme aşamasıdır. Bu aşamada sorunun ne olduğu, çözüm için hangi bilgilere ihtiyaç duyulduğu, bu bilgilerle neler yapılacağı ve sonunda hangi sonucun elde edilmek istendiği belirlenir. Örneğin okul kantinindeki sıra sorununda öğrenci sayısı ve alışveriş süreleri incelenir. Daha sonra bekleme süresini azaltacak bir sıra düzeni planlanır.

### 1.1.1. Algoritma ve Tasarım Kriterleri

Algoritma, belirli bir problemi çözmek veya hedeflenen bir amaca ulaşmak için tasarlanan, açık ve sıralı adımlardır. Algoritmadaki adımlar açık olmalı, sırayla uygulanmalı ve bir sonuca ulaşmalıdır.

**Örnek:** İki sayının toplamını bulma;

1. Birinci sayıyı al.
2. İkinci sayıyı al.
3. İki sayıyı topla.
4. Sonucu ekrana yazdır.

Örneğin girilen sayılar 8 ve 5 ise, ekranda 13 sonucu gösterilir.

### 1.1.2. Problemi Bileşenlerine Ayırma

Problemi Bileşenlerine Ayırma, büyük ve karmaşık bir problemi daha küçük ve yönetilebilir parçalara bölme yöntemidir. Her parça ayrı ayrı incelenir ve çözülür. Bu yöntem, hataları bulmayı, sistemi geliştirmeyi ve güncellemeyi kolaylaştırır.

**Örnek: Dijital oyun tasarlama :** Oyunun karakter hareketleri, sesleri ve puan sistemi ayrı bölümler olarak hazırlanır. Bu bölümler daha sonra birleştirilir. Böylece oyunu geliştirmek ve hataları bulmak kolaylaşır.

### 1.1.3. Girdi-İşlem-Çıktı Modeli

Bilgisayarda problemler genellikle Girdi–İşlem–Çıktı modeliyle çözülür.

**Girdi:** Sisteme verilen bilgiler veya verilerdir.

**İşlem:** Bu bilgiler üzerinde yapılan hesaplama ve kontrollerdir.

**Çıktı:** İşlem sonunda elde edilen sonuçtur.

**Örnek:** Bankamatikten para çekme Kullanıcının kartı, şifresi ve çekmek istediği para miktarı girdidir. Bankamatik şifreyi kontrol eder ve yeterli bakiye olup olmadığına bakar; bu bölüm işlemdir. Para, makbuz ve ekranda gösterilen yeni bakiye ise çıktıdır.

# 1.2. Akış Şeması (Flowchart)

Akış şeması, bir algoritmanın kod yazılmadan önce semboller ve çizimler kullanılarak gösterilmesidir. Çözüm adımlarını daha kolay anlamayı, kontrol etmeyi ve hataları kodlamaya başlamadan fark etmeyi sağlar.

## 1.2.1. Sözde Kod ve Elle Çalıştırma

**Sözde kod,** bir algoritmayı herhangi bir programlama dilinin kurallarına bağlı kalmadan, günlük dile yakın ve açık komutlarla yazma yöntemidir. Sözde kodlar, programcının mantık hatalarına odaklanmasını kolaylaştırır.

**Dry run(Elle Çalıştırma),** bir algoritmayı kodlamadan önce kağıt üzerinde adım adım çalıştırma yöntemidir. Bu sırada değişkenlerin değerleri takip edilir ve sonucun doğru olup olmadığı kontrol edilir. Böylece mantık hataları erken fark edilir.

### 1.2.1.1. Örnek 1: İki Sayının Toplamını Bulma (Sayfa 25 Örnek 1)

Kullanıcı tarafından girilen iki sayının toplamını ekrana yazdıran sözde kodu ve elle çalıştırma tablosunu hazırlayın. Birinci ve ikinci sayı için kullanıcıdan alınan değerler için sırasıyla **s1** ve **s2** değişkenlerini, toplam için **toplam** değişkenini kullanın.

**Sözde Kod**

1. Adım: BAŞLA.
2. Adım: Birinci sayıyı (**s1**) kullanıcıdan al.
3. Adım: İkinci sayıyı (**s2**) kullanıcıdan al.
4. Adım: **toplam = s1 + s2** işlemini yap.
5. Adım: toplam **sonucunu ekrana yazdır**.
6. Adım: DUR.

**Elle Çalıştırma Tablosu**

**Örnek Girdiler:** s1 = 15, s2 = 25 **Beklenen Çıktı:** toplam = 40

| Adım | İşlem | s1 | s2 | toplam | Açıklama |
|---|---|---|---|---|---|
| 1 | BAŞLA | - | - | - | Algoritma başlar |
| 2 | **s1** değerini oku. | **15** | - | - | Kullanıcı 15 sayısını girer |
| 3 | **s2** değerini oku. | 15 | **25** | - | Kullanıcı 25 sayısını girer |
| 4 | **toplam = s1 + s2** | 15 | 25 | **40** | 15 + 25 = 40 işlemi gerçekleşir |
| 5 | toplam değerini yazdır | 15 | 25 | 40 | Ekrana 40 sonucu yazılır |
| 6 | DUR | 15 | 25 | 40 | Algoritma başarıyla sonlanır |

### 1.2.1.2. Karşılaştırma Operatörleri ve Kullanım Örnekleri

Algoritmalar; günlük hayatta olduğu gibi belirli koşullara göre farklı yollara sapabilir. Yazılımcılar, iki veriyi birbiriyle kıyaslayan ve aralarındaki mantıksal ilişkiyi belirleyen **karşılaştırma operatörlerini** kullanarak **Doğru**(True) veya **Yanlış**(False) mantıksal sonucunu hesaplar.

Aşağıda ki tabloda sık karşılaştırma operatörleri ve kullanımı gösterilmektedir.

| Operatör | Anlamı | Örnek Kullanım | Bilgisayar Dilindeki Karşılığı ve Açıklama |
|---|---|---|---|
| == | Eşittir | tahmin == 1234 | **tahmin** değeri, **1234** sayısına eşitse sonuç **Doğru** olur. |
| != | Eşit Değildir | sifre != “123” | Şifre, “123” değerine eşit değilse sonuç **Doğru** olur. |
| > | Büyüktür | yas > 18 | Yaş değeri, 18’den büyükse sonuç **Doğru** olur. |
| < | Küçüktür | sicaklik < 0 | Sıcaklık, 0’dan küçükse sonuç **Doğru** olur. |
| >= | Büyük Eşittir | puan >= 50 | Puan, 50 veya 50’den daha büyükse sonuç **Doğru** olur. |
| <= | Küçük Eşittir | hiz <= 90 | Hız, 90 veya 90’dan daha küçükse sonuç **Doğru** olur. |

#### 1.2.1.2.1. Karşılaştırma Operatörleri İle Yapılan İşlem Örnekleri

**EŞİTTİR (==)**
- **5 == 5** → Sonuç: true (1)
- **12 == 20** → Sonuç: false (0)
- **“Tuna” == “Tuna”** → Sonuç: true (1)
- **“Ahmet” == “ahmet”** → Sonuç: false (0) (Büyük/küçük harf duyarlılığı)
- **–10 == 10** → Sonuç: false (0)

**EŞİT DEĞİLDİR (!=)**
- **10 != 5** → Sonuç: true (1)
- **7 != 7** → Sonuç: false (0)
- **“Java” != “Python”** → Sonuç: true (1)
- **“Okul” != “Okul”** → Sonuç: false (0)
- **–4 != 4** → Sonuç: true (1)

**BÜYÜKTÜR (>)**
- **15 > 8** → Sonuç: true (1)
- **3 > 10** → Sonuç: false (0)
- **5 > 5** → Sonuç: false (0)
- **–2 > –10** → Sonuç: true (1)
- **0.5 > 0.2** → Sonuç: true (1)

**KÜÇÜKTÜR (<)**
- **4 < 9** → Sonuç: true (1)
- **25 < 12** → Sonuç: false (0)
- **7 < 7** → Sonuç: false (0)
- **–15 < –3** → Sonuç: true (1)
- **1.25 < 1.30** → Sonuç: true (1)

**BÜYÜK EŞİTTİR (>=)**
- **18 >= 18** → Sonuç: true (1)
- **20 >= 15** → Sonuç: true (1)
- **6 >= 10** → Sonuç: false (0)
- **0 >= 0** → Sonuç: true (1)
- **–5 >= –5** → Sonuç: true (1)

**KÜÇÜK EŞİTTİR (<=)**
- **5 <= 5** → Sonuç: true (1)
- **3 <= 8** → Sonuç: true (1)
- **14 <= 9** → Sonuç: false (0)
- **–8 <= –2** → Sonuç: true (1)
- **100 <= 99** → Sonuç: false (0)

### 1.2.1.3. Örnek 1 Hatalı Giriş Kontrolü Algoritması (Sayfa 28 Örnek 4)

Kullanıcının önceden belirlediği bir şifre ile sisteme girişini kontrol eden programının sözde kodunu ve elle çalıştırma tablosunu hazırlayın. Sistem şifresi için **sistemKey**, kullanıcı şifresi için **kullaniciKey** kullanılacak.

**Sözde Kod**

1. Adım: BAŞLA
2. Adım: **sistemKey = 1234**
3. Adım: kullanıcıdan (**kullaniciKey**) değerini al
4. Adım: **EĞER sistemKey == kullaniciKey**
   **EVET** ise "Sisteme Girer"
   **HAYIR** ise "Sisteme Giremez!"
5. Adım: BİTİR

**Elle Çalışma Tablosu**

**Örnek Girdiler:** sistemKey=1234, kullaniciKey=9876, **Beklenen Çıktı:** Sisteme Giremez

| Ad | İşlem | sistemKey | kullaniciKey | Koşul sistemKey == kullaniciKey | Çıktı | Açıklama |
|---|---|---|---|---|---|---|
| 1 | BAŞLA | - | - | - | - | Algoritma başlar. |
| 2 | sistemKey = 1234 | **1234** | - | - | - | Kodun içinde sistem şifresi **sistemKey=1234** oluşturuldu |
| 3 | kullaniciKey = 9876 Gir | 1234 | **9876** | - | - | Kullanıcıdan **kullaniciKey=9876** değeri alındı. |
| 4 | sistemKey == kullaniciKey | 1234 | 9876 | **HAYIR** | **Giriş yapılamaz!** | Sistemde bulunan şifre ile kullanıcının girdiği şifre karşılaştırılır. Kullanıcı sisteme giremez! |
| 5 | DUR | 1234 | 9876 | - | - | Algoritma başarıyla sonlanır. |

**Elle Çalışma Tablosu**

**Örnek Girdiler:** sistemKey=1234, kullaniciKey=1234, **Beklenen Çıktı:** Sisteme Girer

| Ad | İşlem | sistemKey | kullaniciKey | Koşul sistemKey == kullaniciKey | Çıktı | Açıklama |
|---|---|---|---|---|---|---|
| 1 | BAŞLA | - | - | - | - | Algoritma başlar. |
| 2 | sistemKey = 1234 | **1234** | - | - | - | Kodun içinde sistem şifresi **sistemKey=1234** oluşturuldu |
| 3 | kullaniciKey = 1234 Gir | 1234 | **1234** | - | - | Kullanıcıdan **kullaniciKey=1234** değeri alındı. |
| 4 | sistemKey == kullaniciKey | 1234 | 1234 | **EVET** | **Giriş yapılır.** | Sistemde bulunan şifre ile kullanıcının girdiği şifre karşılaştırılır. Kullanıcı sisteme girer. |
| 5 | DUR | 1234 | 1234 | - | - | Algoritma başarıyla sonlanır. |

### 1.2.1.4. Örnek 2 Sıcaklık Kontrolü (Sayfa 28 Örnek 3)

Ortam sıcaklığı (**SICAKLIK**) 20 derecenin altına düştüğünde kombiyi çalıştıran, 20 derecenin üzerine çıktığında ise kombiyi durduran bir termostat programının sözde kodunu ve elle çalıştırma tablosunu hazırlayın.

**Sözde Kod**

1. Adım: BAŞLA
2. Adım: Sıcaklık değeri oku **SICAKLIK**
3. Adım: **EĞER SICAKLIK < 20**
   **EVET** ise "Kombi Çalıştır"
   **HAYIR** ise "Kombi Durdu"
4. Adım: BİTİR

**Elle Çalışma Tablosu**

**Örnek Girdiler:** SICAKLIK = 18, **Beklenen Çıktı:** Kombi Çalışır

| Adım | İşlem | SICAKLIK | Koşul (SICAKLIK < 20) | Çıktı | Açıklama |
|---|---|---|---|---|---|
| 1 | BAŞLA | - | - | - | Algoritma başlar. |
| 2 | SICAKLIK oku | **18** | - | - | Ortamdaki sıcaklık değeri okunur, SICAKLIK değerine atanır |
| 3 | SICAKLIK < 20 | 18 | **EVET** | **Kombi Çalışır!** | Koşul Evet |
| 4 | DUR | 18 | - | - | Algoritma başarıyla sonlanır. |

**Örnek Girdiler:** SICAKLIK = 22, **Beklenen Çıktı:** Kombi Durur

| Adım | İşlem | SICAKLIK | Koşul (SICAKLIK < 20) | Çıktı | Açıklama |
|---|---|---|---|---|---|
| 1 | BAŞLA | - | - | - | Algoritma başlar. |
| 2 | SICAKLIK oku | **20** | - | - | Ortamdaki sıcaklık değeri okunur, SICAKLIK değerine atanır |
| 3 | SICAKLIK < 20 | 20 | **HAYIR** | **Kombi Durur!** | Koşul Hayır |
| 4 | DUR | 18 | - | - | Algoritma başarıyla sonlanır. |

### 1.2.1.5. Örnek 3: Yaş Kontrolü ve Ehliyet Alabilme (Sayfa 25 Örnek 2)

Kullanıcıdan o günün yılını (**sistemYil**) ve kullanıcıdan doğum yılını (**dogumYil**) alarak kullanıcının 18 yaşından büyük olması durumunda "**Ehliyet Alır!**" küçük olması durumunda "**Ehliyet Almaz!**" yazısını ekrana yazdıran programın sözde kodunu ve elle çalışma tablosunu hazırlayınız.

**Sözde Kod**

1. Adım: BAŞLA.
2. Adım: Sistem Tarihi (**sistemYil**) değerini oku.
3. Adım: Kullanıcının Doğum Yılı (**dogumYil**) değerini oku.
4. Adım: **yas = sistemYil - dogumYil** işlemini yap.
5. Adım: **Eğer yas >= 18**
   **EVET** ise "**Ehliyet Alır!**" mesajını yazdır.
   **HAYIR** ise "**Ehliyet Almaz!**" mesajını yazdır.
6. Adım: DUR.

**Elle Çalışma Tablosu**

**Örnek Girdiler:** Sistem Tarihi (sistemYil) = 2026, Doğum Yılı (dogumYil) = 2005, **Beklenen Çıktı:** Ehliyet alabilirsiniz.

| Adım | İşlem | sistemYil | dogumYil | yas | Koşul (yas >= 18) | Çıktı | Açıklama |
|---|---|---|---|---|---|---|---|
| 1 | BAŞLA | - | - | - | - | - | Algoritma başlar. |
| 2 | sistemYil oku | **2026** | - | - | - | - | Sistem tarihi alınır. |
| 3 | dogumYil oku | 2026 | **2005** | - | - | - | Kullanıcı 2005 sayısını girer. |
| 4 | yas =sistemYil - dogumYil | 2026 | 2005 | **21** | - | - | 2026 - 2005 = 21 |
| 5 | yas >= 18 | 2026 | 2005 | 21 | **Evet** | **Ehliyet Alır!** | Koşul doğru → mesaj yazdırılır. |
| 6 | DUR | 2026 | 2005 | 21 | - | - | Algoritma başarıyla sonlanır. |

**Örnek Girdiler:** Sistem Tarihi (sistemYil) = 2026, Doğum Yılı (dogumYil) = 2010, **Beklenen Çıktı:** Ehliyet almak için henüz küçüksünüz.

| Adım | İşlem | sistemYil | dogumYil | yas | Koşul (yas >= 18) | Çıktı | Açıklama |
|---|---|---|---|---|---|---|---|
| 1 | BAŞLA | - | - | - | - | - | Algoritma başlar. |
| 2 | sistemYil oku | **2026** | - | - | - | - | Sistem tarihi alınır. |
| 3 | dogumYil oku | 2026 | **2010** | - | - | - | Kullanıcı 2010 sayısını girer. |
| 4 | yas =sistemYil - dogumYil | 2026 | 2010 | **16** | - | - | 2026 - 2010 = 16 |
| 5 | yas >= 18 | 2026 | 2010 | 16 | **Hayır** | **Ehliyet Almaz!** | Koşul yanlış → mesaj yazdırılır. |
| 6 | DUR | 2026 | 2010 | 16 | - | - | Algoritma başarıyla sonlanır. |

## 1.2.1.6. Flowgorithm ve Thonny Uygulamaları

Derslerimizde kullanacağımız yazılımların kurulum bağlantılarına ve destekleyici eğitim videolarına yan taraftaki karekodları taratarak veya altlarındaki bağlantılara tıklayarak doğrudan erişebilirsiniz.

**Flowgorithm Programı**

Algoritmik düşünme becerimizi geliştirmek ve tasarladığımız mantıksal adımları görselleştirmek için Flowgorithm yazılımını kullanacağız. Bu program sayesinde yazdığımız akış şemalarını adım adım çalıştırabilir, değişkenlerin değer değişimlerini anlık olarak izleyebilir ve algoritmalarımızı kod yazmadan önce test edebiliriz.

**Flowgorithm Programı**

Bağlantıya tıklayın
flowgorithm.org

**Flowgorithm Youtube Eğitimi**

Derste işlediğimiz konuları pekiştirmek ve kendi hızınızda pratik yapabilmeniz için Atilla Çetin tarafından hazırlanan Flowgorithm ile Algoritma ve Akış Diyagramları video serisinden yararlanabilirsiniz. Temel kavramlardan uygulamalı örneklere kadar uzanan bu oynatma listesine sağdaki karekodu okutarak veya bağlantıya tıklayarak ulaşabilirsiniz.

**Youtube Eğitimi**

Bağlantıya tıklayın
kisa.link/bHGEg

**Thonny IDE**

Akış şemalarından kodlamaya geçerken, Python öğrenimi için özel olarak tasarlanmış Thonny IDE'yi kullanacağız. Sade arayüzü ve adım adım kod takibi sunan hata ayıklama (debug) özellikleriyle başlangıç seviyesine son derece uygundur. Dahili Python desteği sayesinde harici bir kurulum gerektirmez; yalnızca Thonny'yi kurarak kodlarınızı hemen çalıştırmaya başlayabilirsiniz.

**Thonny**

Bağlantıya tıklayın
thonny.org

**Programlamanın Temelleri Ders Notları**

Derslerimizde işlediğimiz konuları düzenli takip edebilmeniz ve tekrar yapabilmeniz için hazırladığım bu doküman; MEB'in 14 Eylül 2026 tarihli **'Programlamanın Temelleri 9'** müfredatını temel almaktadır. Konu anlatımları, akış şemaları ve örnek kod uygulamalarıyla ders içi kazanımları pekiştirmeyi amaçlayan kapsamlı bir çalışma kaynağıdır

**Ders Notları**

Bağlantıya tıklayın
kisa.link/hpYCg

### 1.2.1.7. Değişkenler ve Veri Tipleri

**DEĞİŞKEN NEDİR?**

Programın çalışması sırasında kullanıcıdan alınan, hesaplanan veya geçici olarak hafızada (RAM) tutulması gereken verileri saklayan isimlendirilmiş bellek alanlarıdır.

Flowgorithm üzerinde bir değişkeni kullanmadan önce mutlaka **Tanımlama (Declare)** adımıyla bilgisayara tanıtmak gerekir. Aynı türdeki birden fazla değişken aralarına virgül konularak tek seferde tanımlanabilir (Örn: `sistemYil, dogumYil, yas`).

#### 1.2.1.7.1. Temel Veri Tipleri

Flowgorithm içerisinde en sık kullanılan 4 temel veri tipi bulunur:

| Veri Tipi | Türü | Örnek Değerler | Açıklama |
|---|---|---|---|
| **Tamsayı** (Integer) | Tam Sayı | `15`, `0`, `-8`, `2026` | Küsüratsız, negatif veya pozitif tam sayı değerlerini tutar. Yaş, adet, sayaç gibi veriler için kullanılır. |
| **Reel** (Real / Float) | Ondalıklı Sayı | `3.14`, `18.5`, `-0.75` | Kesirli veya ondalıklı sayıları tutar. Ortalama, boy, sıcaklık veya fiyat gibi hassas hesaplamalarda tercih edilir. |
| **String** | Metinsel Veri | `"Ahmet"`, `"1234"`, `"A"` | Çift tırnak içinde yazılan her türlü yazı, harf, sembol ve rakamı saklar. **Matematiksel işlemlere girmez.** |
| **Boolean** | Mantıksal | `true(1)`, `false(0)` | Yalnızca iki değerden birini alabilir: **Doğru (true(1))** ya da **Yanlış (false(0))**. Karar ve döngü kontrollerinde kullanılır. |

#### 1.2.1.7.2. Değişken İsimlendirme Kuralları

Değişken tanımlarken programların hata vermemesi ve kodun düzenli olması için belirli kurallara uymak şarttır.

**Türkçe Karakter Kullanımı**

Kodlama dillerinin ve yazılım araçlarının küresel standardı gereği, değişken isimlendirirken
**Türkçe'ye özgü harfleri kullanmaktan kesinlikle kaçınınız!**.
Aşağıdaki tabloda büyük ve küçük Türkçe karakterlerin yerine İngilizce alfabe karşılıkları tercih edilmelidir:

| Türkçe Karakter | Doğru Kullanım | İngilizce Karakte |
|---|---|---|
| **ç / Ç** | → | **c / C** |
| **ğ / Ğ** | → | **g / G** |
| **ı / İ** | → | **i / I** |
| **ö / Ö** | → | **o / O** |
| **ş / Ş** | → | **s / S** |
| **ü / Ü** | → | **u / U** |

**Camel Case (Deve Notasyonu) Kuralı**

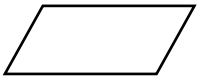
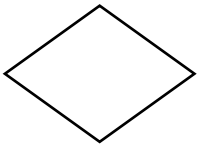
Değişken adları tanımlanırken kelimeler arasında boşluk bırakılamaz. Birden fazla kelimeden oluşan değişkenleri okunaklı kılmak için dünya çapında standart olan **camelCase** yöntemi kullanılır.
**Kural:** İlk kelimenin tüm harfleri küçük başlar. Sonraki her kelimenin ilk harfi büyük yazılır (kelimelerin ortasındaki hörgüç görünümü gibi).

| Yanlış Biçim | camelCase | Açıklama |
|---|---|---|
| `sistem yil` | `sistemYil` | Boşluk yerine birleşik yazılır |
| `dogumyil` | `dogumYil` | İkinci kelimenin baş harfi büyük |
| `ogrencinotortalamasi` | `ogrenciNotOrtalamasi` | Okunabilirlik artırılır |

## 1.2.2. Akış Şeması (Flowchart) Sembolleri ve Görsel Modelleme

**Akış şeması(flowchat)**, bir algoritmanın adımlarını semboller ve oklarla gösteren görsel bir modeldir. Programın çalışma mantığını kolayca anlamayı ve kodlama öncesinde olası hataları fark etmeyi sağlar.

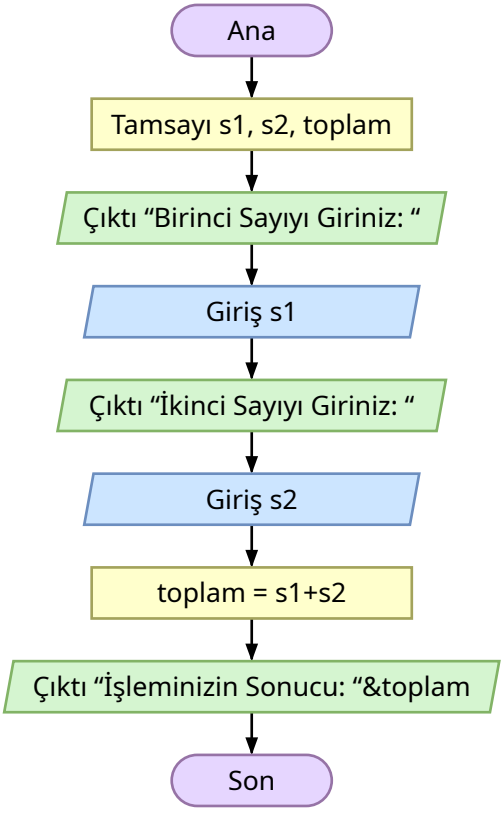
**Tablo 1.2:** Temel Akış Şeması Sembolleri, İşlevleri ve Günlük Hayat Karşılıkları.

| Sembol | Adı | İşlevi | Günlük Hayat Karşılığı |
|---|---|---|---|
|  | Oval (Terminal) | Algoritmanın başlangıcını ve bitişini temsil eder. | Bir işe başlama veya sonlandırma anıdır. |
|  | Paralelkenar (Giriş / Çıkış) | Kullanıcıdan veri alınacağını veya ekrana bilgi yazdırılacağını belirtir. | Soru sormak veya cevap vermek içindir. |
|  | Dikdörtgen (İşlem) | Matematiksel hesaplamalar ve değişken atamaları için kullanılır. | Bir eylemi gerçekleştirmek içindir. |
|  | Eşkenar Dörtgen (Karar) | Bir koşulun kontrol edildiği noktadır. Sonuca göre akış Evet veya Hayır yönünde ilerler. | Yol ayrımında karar vermek içindir. |
|  | Oklar (Akış Yönü) | Adımlar arasındaki ilerleme yönünü gösterir. | İzlenecek yolu gösteren oklardır. |

### 1.2.2.1. Örnek 1: İki Sayının Toplamını Bulma (Sayfa 25 Örnek 1)

Kullanıcı tarafından girilen iki sayının toplamını bulan örneğin akış şemasını (flowchart) çiziniz. Birinci ve ikinci sayı ve toplam için sırasıyla **s1**, **s2**, **toplam** değişkenleri kullanılacak.

**Akış Şeması(Flowchart)**

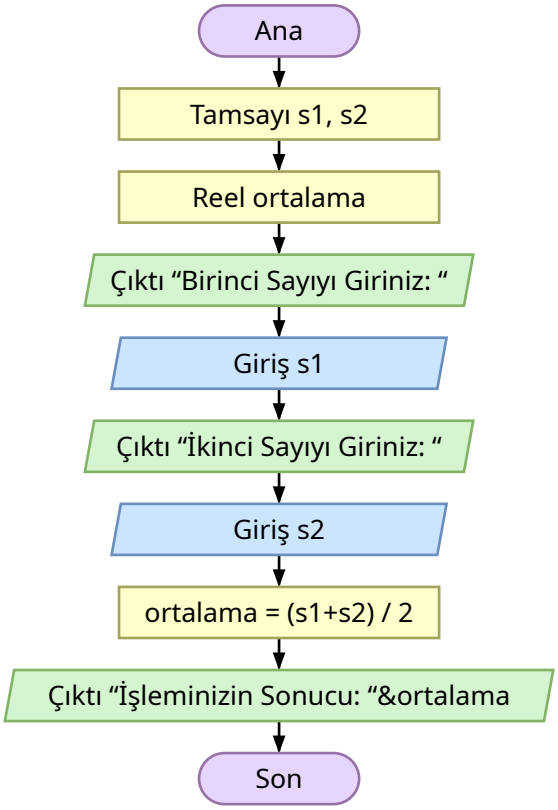



### 1.2.2.2. Örnek 2: İki Sayının Ortalamasını Bulma

Kullanıcı tarafından girilen iki sayının ortalamasını bulan örneğin akış şemasını(flowchat) çiziniz. Birinci ve ikinci sayı ve ortalama için sırasıyla **s1**, **s2**, **ortalama** değişkenleri kullanılacak.

Ortalama hesabı: `ortalama = (s1 + s2) / 2`

**Akış Şeması(Flowchart)**

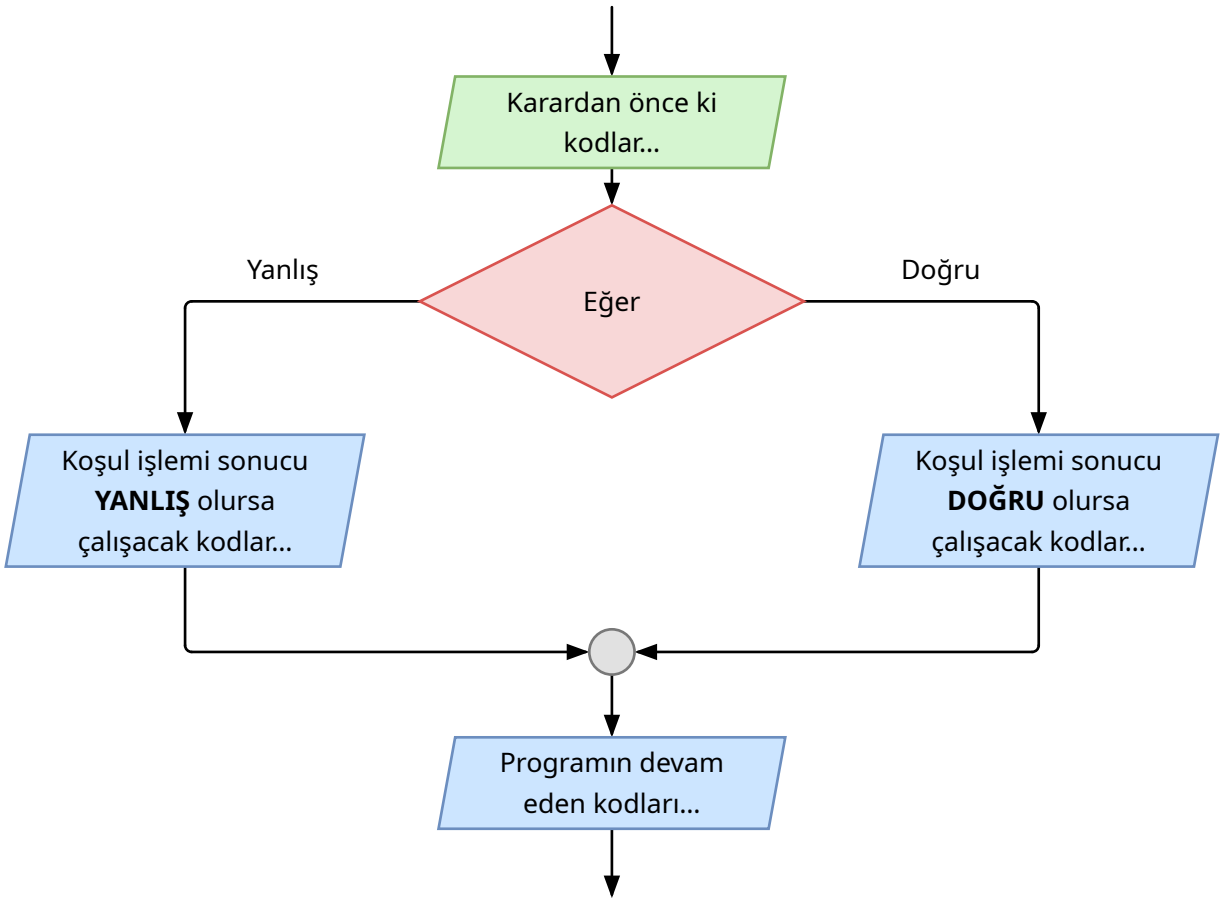


### 1.2.2.3. Örnek Çalışma Soruları

1. Yaş Hesaplama: Kullanıcıdan doğum yılını ve mevcut yılı alıp yaşını hesaplayan ve ekrana yazdıran akış şemasını çiziniz.
2. Kare Alan ve Çevre: Bir kenar uzunluğu kullanıcıdan girilen karenin alanını ve çevresini hesaplayıp ekrana yazdıran akış şemasını çiziniz.
3. KDV Hesaplama: Fiyatı girilen bir ürünün %20 KDV tutarını ve KDV dahil toplam satış fiyatını hesaplayıp ekrana yazdıran akış şemasını çiziniz.
4. Birim Dönüşümü (Sıcaklık): Kullanıcıdan Celsius cinsinden sıcaklık değerini alıp Fahrenheit değerine çeviren (F = C *1.8 + 32) ve sonucu gösteren akış şemasını çiziniz.
5. Taksi Ücreti Hesaplama: Taksimetre açılış ücreti 30 TL, kilometre başına ücret 20 TL olan bir şehirde; gidilen kilometre bilgisini kullanıcıdan alıp ödenecek toplam tutarı hesaplayıp yazdıran akış şemasını çiziniz.

### 1.2.2.4. Karar Yapıları

Karar yapıları, programın bir koşulu kontrol ederek hangi adımı izleyeceğini belirler. Akış şemasında eşkenar dörtgen ile gösterilir. Koşul doğruysa bir yola, yanlışsa başka bir yola devam edilir. Altta ki şemada görüldüğü gibi eğer işlemi ile yapılan kontrolün sonunda kod doğru ya da yanlış ihtimale doğru hareket edecektir.



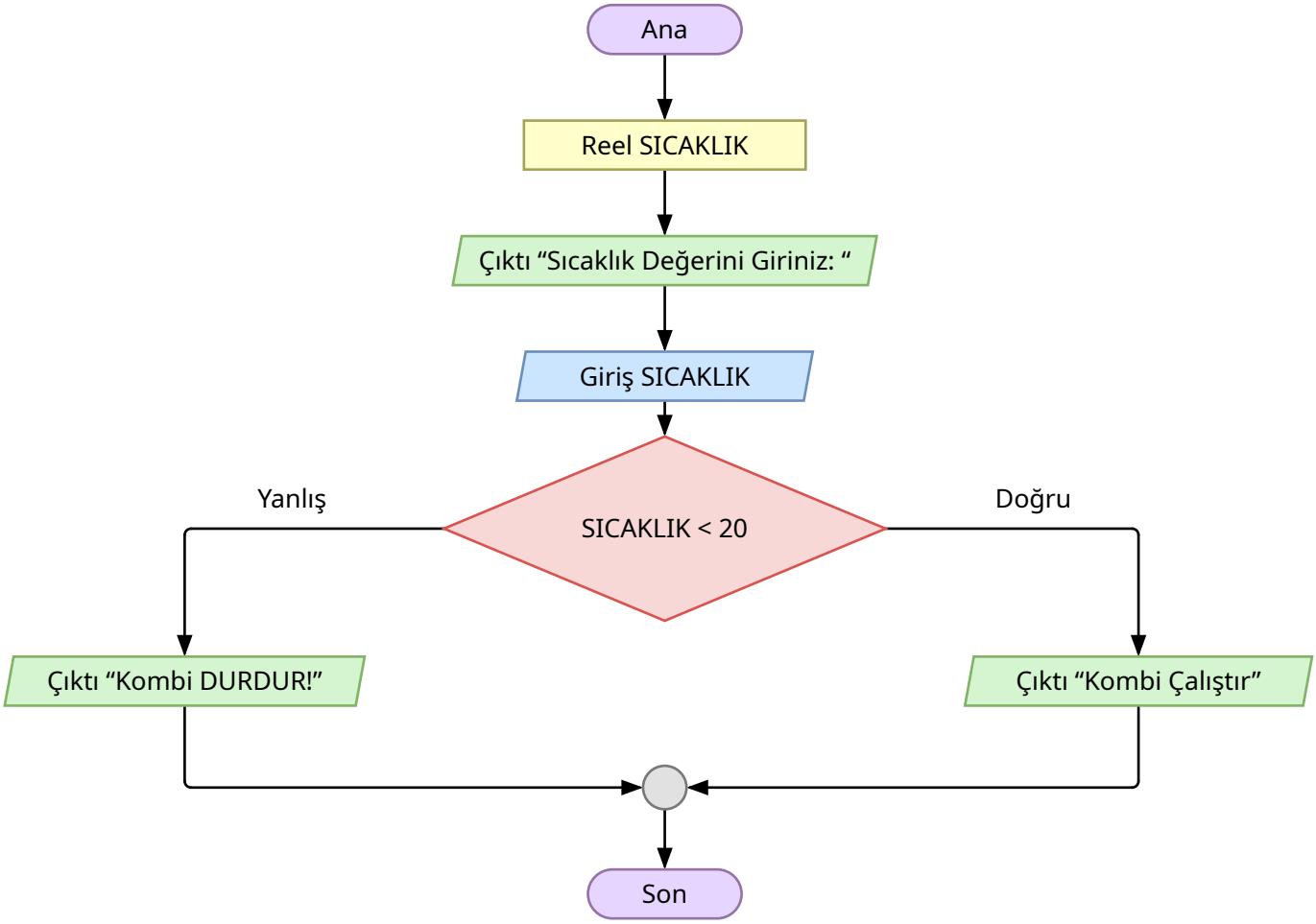
### 1.2.2.5. Örnek 3 Sıcaklık Kontrolü

Ortam sıcaklığı (**SICAKLIK**) 20 derecenin altına düştüğünde kombiyi çalıştıran, 20 derecenin üzerine çıktığında ise kombiyi durduran bir termostat programının akış şemasını hazırlayın.

**Sözde Kod**

1. Adım: BAŞLA
2. Adım: Sıcaklık değeri oku **SICAKLIK**
3. Adım: **EĞER SICAKLIK < 20**
   **EVET** ise “Kombi Çalıştır”
   **HAYIR** ise “Kombi Durdu”
4. Adım: BİTİR

**Akış Şeması(Flowchart)**



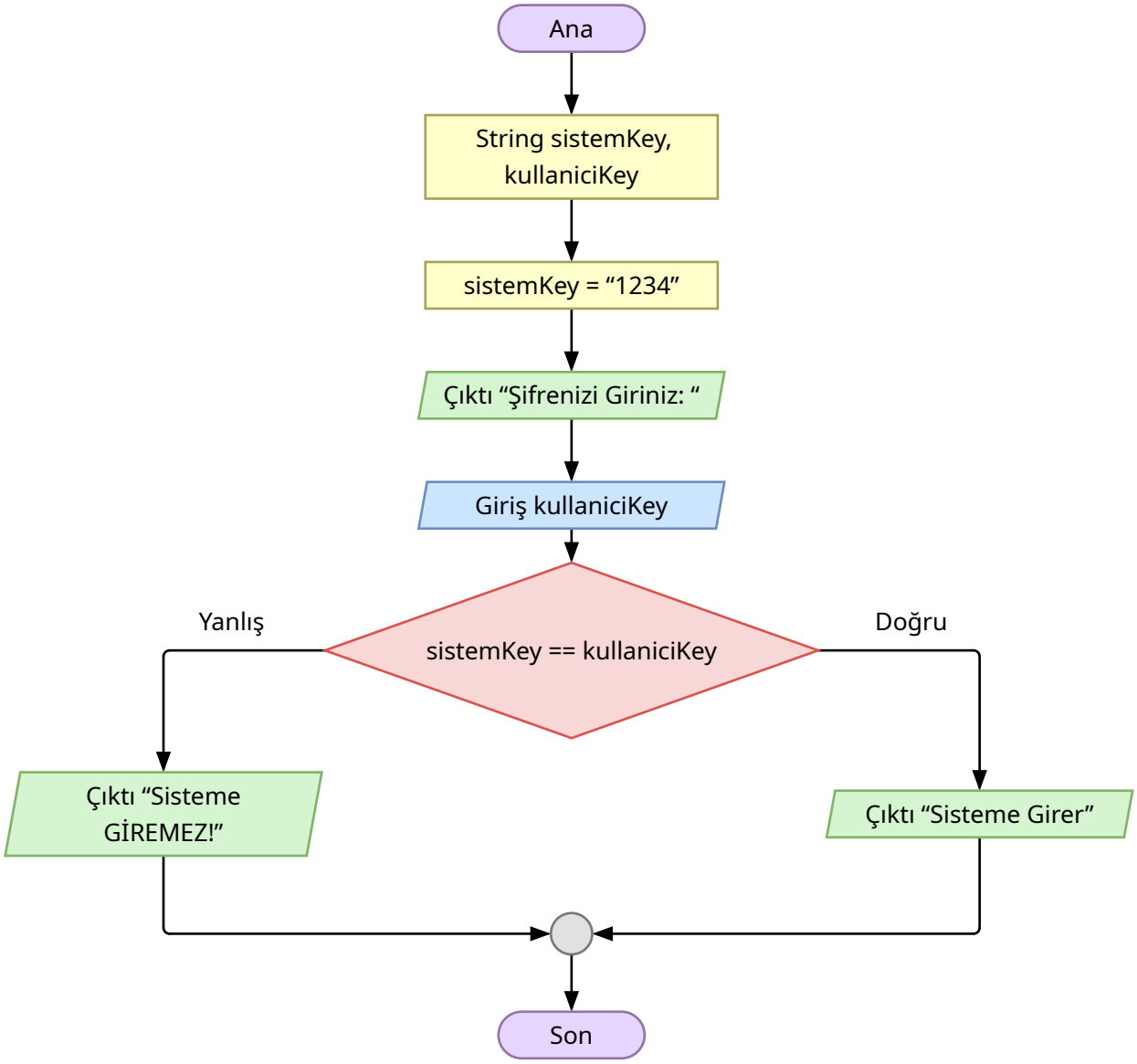
### 1.2.2.6. Örnek 4 Hatalı Giriş Kontrolü Algoritması

Kullanıcının önceden belirlediği bir şifre ile sisteme girişini kontrol eden programının akış şemasını hazırlayın. Sistem şifresi için **sistemKey**, kullanıcı şifresi için **kullaniciKey** kullanılacak.

**Sözde Kod**

1. Adım: BAŞLA
2. Adım: **sistemKey = 1234**
3. Adım: kullanıcıdan (**kullaniciKey**) değerini al
4. Adım: **EĞER sistemKey == kullaniciKey**
   **EVET** ise “Sisteme Girer”
   **HAYIR** ise “Sisteme Giremez!”
5. Adım: BİTİR

**Akış Şeması(Flowchart)**



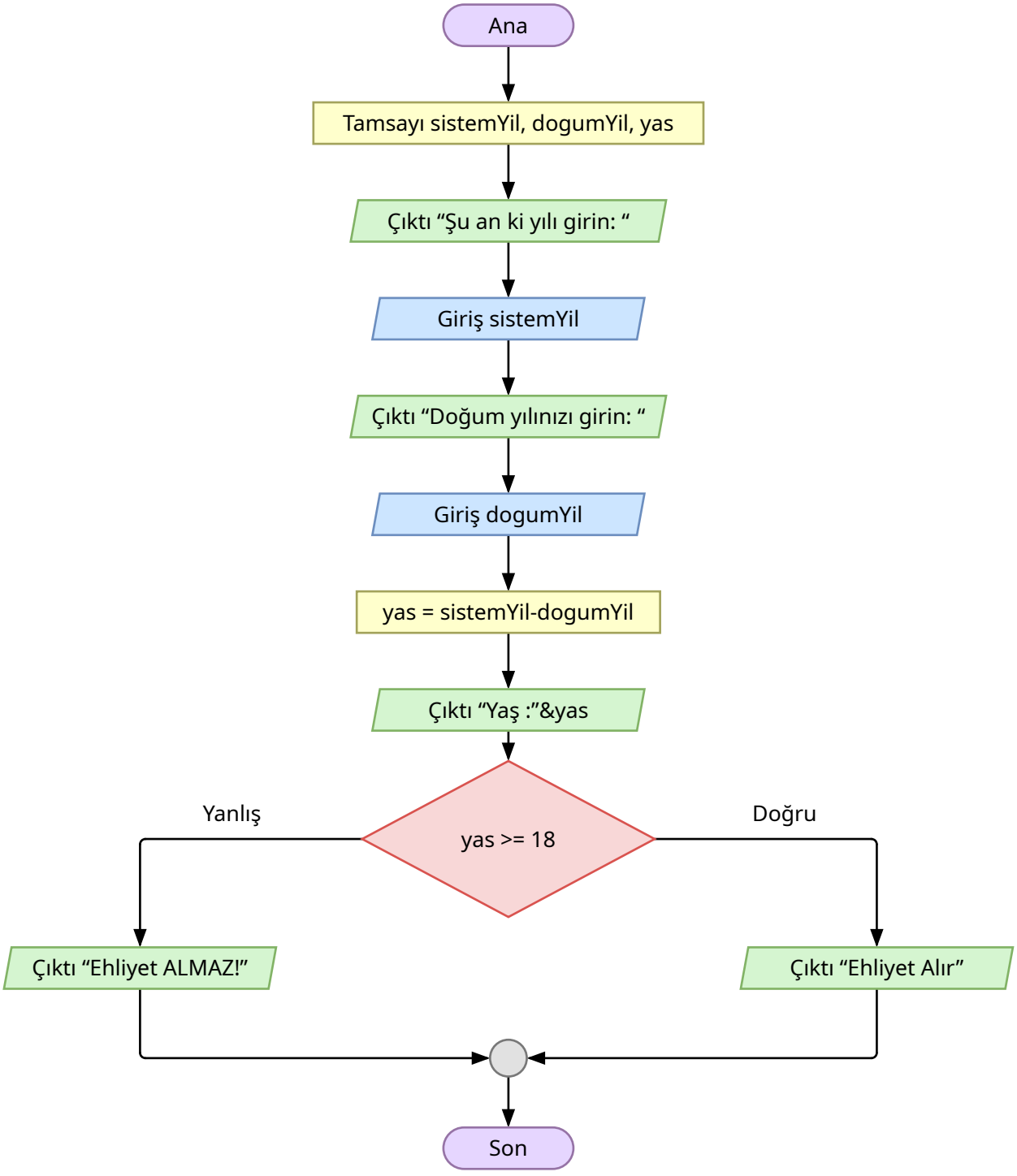
### 1.2.2.7. Örnek 5: Yaş Kontrolü ve Ehliyet Alabilme

Kullanıcıdan o günün yılını (**sistemYil**) ve kullanıcıdan doğum yılını (**dogumYil**) alarak kullanıcının 18 yaşından büyük olması durumunda "**Ehliyet Alır!**" küçük olması durumunda "**Ehliyet Almaz!**" yazısını ekrana yazdıran programın sözde kodunu ve elle çalışma tablosunu hazırlayınız.

**Sözde Kod**

2. Adım: Sistem Tarihi (**sistemYil**) değerini oku.
3. Adım: Kullanıcının Doğum Yılı (**dogumYil**) değerini oku.
4. Adım: **yas = sistemYil - dogumYil** işlemini yap.
5. Adım: **Eğer yas >= 18**
   **EVET** ise "**Ehliyet Alır!**" mesajını yazdır.
   **HAYIR** ise "**Ehliyet Almaz!**" mesajını yazdır.

**Akış Şeması(Flowchart)**



### 1.2.2.8. Örnek Çalışma Soruları Karar Yapıları

1. Akıllı Bahçe Sulama Sistemi (Sensör ve Eşik Değeri Mantığı): Bir seradaki toprak nem oranı sensöründen gelen değer **nem** değişkeninde tutulmaktadır.
   - Nem oranı 30'un altına düştüğünde sulama vanasını açan (**"Sulama Başlatıldı"**),
   - 30 veya üzerinde olduğunda ise sulama vanasını kapatan (**"Sulama Durduruldu"**) programın akış şemasını çiziniz.
2. Bakiye ve Ödeme Kontrolü (İki Değişkeni Karşılaştırma): Bir online alışveriş sisteminde kullanıcının mevcut hesap bakiyesi **bakiye**, satın almak istediği sepet tutarı ise **sepet** değişkenidir.
   - Eğer **bakiye** değeri **sepet** tutarına eşit veya büyükse ekrana **"Ödeme Başarılı"** yazdıran ve kalan yeni bakiyeyi ekranda gösteren,
   - Yetersiz ise ekrana **"Yetersiz Bakiye"** uyarısı veren programın akış şemasını çiziniz.
3. Bir öğrencinin vize ve final notunu kullanıcıdan alarak, öğrencinin not ortalamasını hesaplayan programın akış şemasını çizin.
   - Ortalama formülü: ortalama = (vize **0.4) + (final** 0.6)
   - Eğer ortalama değeri 50 veya üzerinde ise ekrana **"Geçti"**, 50'nin altında ise **"Kaldı"** yazdırılacak
4. Web sitesi kargo ücreti hesaplama;
   - Sipariş tutarı 500 TL veya üzerinde ise kargo ücretsizdir
   - Sipariş tutarı 500 TL'nin altında ise 50 TL kargo ücreti eklenmektedir.

   Kullanıcıya son ödenecek toplam tutarı ekrana yazdıran programın akış şemasını hazırlayınız.
5. Kullanıcıdan bir tamsayı (**sayi**) girmesini isteyen, girilen sayının 0'dan büyük olması durumunda ekrana **"Pozitif Sayı"**, 0'dan küçük olması durumunda **"Negatif Sayı"** yazdıran programın akış şemasını hazırlayın.

### 1.2.2.9. Döngü(Loop)

Bazı işlemlerin belirli bir koşul sağlanana kadar tekrar edilmesi gerekir. Bu tür işlemler için döngüler kullanılır.

Bir sayaç kontrollü döngü üç temel bölümden oluşur:

- **Başlangıç değeri:** Sayacın başlayacağı değerdir.
- **Döngü koşulu:** Döngünün devam edip etmeyeceğini belirler.
- **Sayaç:** Her tur sonunda değeri artırır veya azaltır.

Koşul doğru olduğu sürece döngü devam eder. Koşul yanlış olduğunda döngü sona erer. Sayaç değerinin değiştirilmesi, döngünün sonsuza kadar sürmesini önler.

### 1.2.2.10. Örnek 6 Döngüsel Ekran Yazımı (Sayfa 37 Örnek 8)

Döngü kullanarak ekrana üç kez “Merhaba Geliştirici!” mesajını yazdırınız.

**Sözde Kod**

1. Adım: **sayac** tam sayı değişkenini tanımla
2. Adım: sayac = 1 ile sayac değişkenine ilk değeri olan biri ata.
3. Adım: Eğer **sayac <= 3** sayac küçük eşit
   **EVET** ise

   **“Merhaba Geliştirici!”** yazdız
   **sayac = sayac + 1** => sayac değerini bir arttır

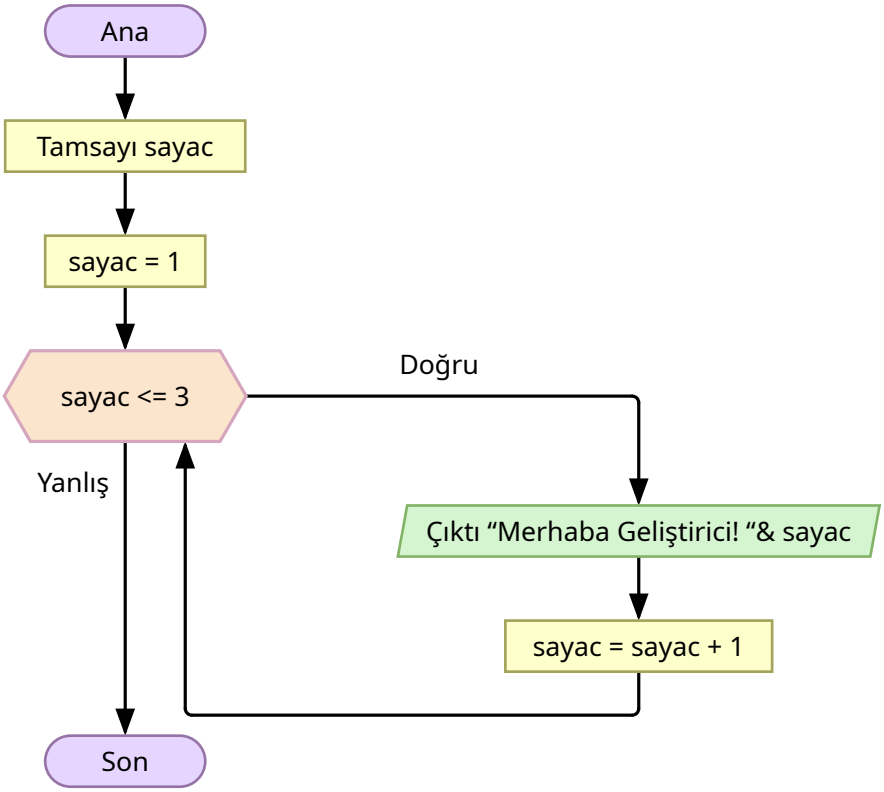
   **HAYIR** ise

   **Döngüden çık**

**Elle Çalışma Tablosu**
**Örnek Girdiler:** sayac = 3 , **Beklenen Çıktı:** ekrana üç kere “Merhaba Geliştirici” yazdırılacak

| Adım | İşlem | sayac | Döngü Koşul (sayac <= 3) | Çıktı | Açıklama |
|---|---|---|---|---|---|
| 1 | Tam sayı **sayac** | - | - | - | sayac değişkeni tanımlanır |
| 2 | sayac = 1 | 1 | - | - | sayac değişkenine 1 değeri atanır |
| 3 | sayac <= 3 | 1 | 1 <= 3 **EVET** | Merhaba Geliştirici | **1<= 3** sonucu **EVET** ekrana yazdırılır sayac 1 arttırılır |
| 4 | sayac <= 3 | 2 | 2<= 3 **EVET** | Merhaba Geliştirici | **2<= 3** sonucu **EVET** ekrana yazdırılır sayac 1 arttırılır |
| 5 | sayac <= 3 | 3 | 3<= 3 **EVET** | Merhaba Geliştirici | **3<= 3** sonucu **EVET** ekrana yazdırılır sayac 1 arttırılır |
| 6 | sayac <= 3 | 4 | 4<= 3 **HAYIR** | - | **4<= 3** sonucu **HAYIR** **Döngü Sonlanır** |

**Akış Şeması (Flowchart)**




**Ekran Çıktısı**

Merhaba Geliştirici! 1

Merhaba Geliştirici! 2

Merhaba Geliştirici! 3

### 1.2.2.11. Örnek 7 Ardışık Sayıların Toplamı (Sayfa 38 Uygulama 4)

Birden başlayarak kullanıcının belirlediği değere kadar belirlenen dahil tüm sayıların toplamını alan kodun akış şemasını hazırlayın. **Örnek Hesap : `n = 3 için toplam = 1 + 2 + 3 = 6`**

**Sözde Kod**

1. Adım: **sayac, toplam ve n** tam sayı değişkenini tanımla
2. Adım: **sayac = 1** ile sayac değişkenine ilk değeri olan biri ata.
3. Adım: **toplam = 0** başlangıç değerini ata
4. Adım: kullanıcıdan **n** değişkenin değerini al
5. Adım: Eğer **sayac <= n** sayac küçük eşit n
   **EVET** ise

   **toplam = toplam + sayac** => toplama işlemini yap
   **sayac = sayac + 1** => sayac değerini bir arttır

   **HAYIR** ise

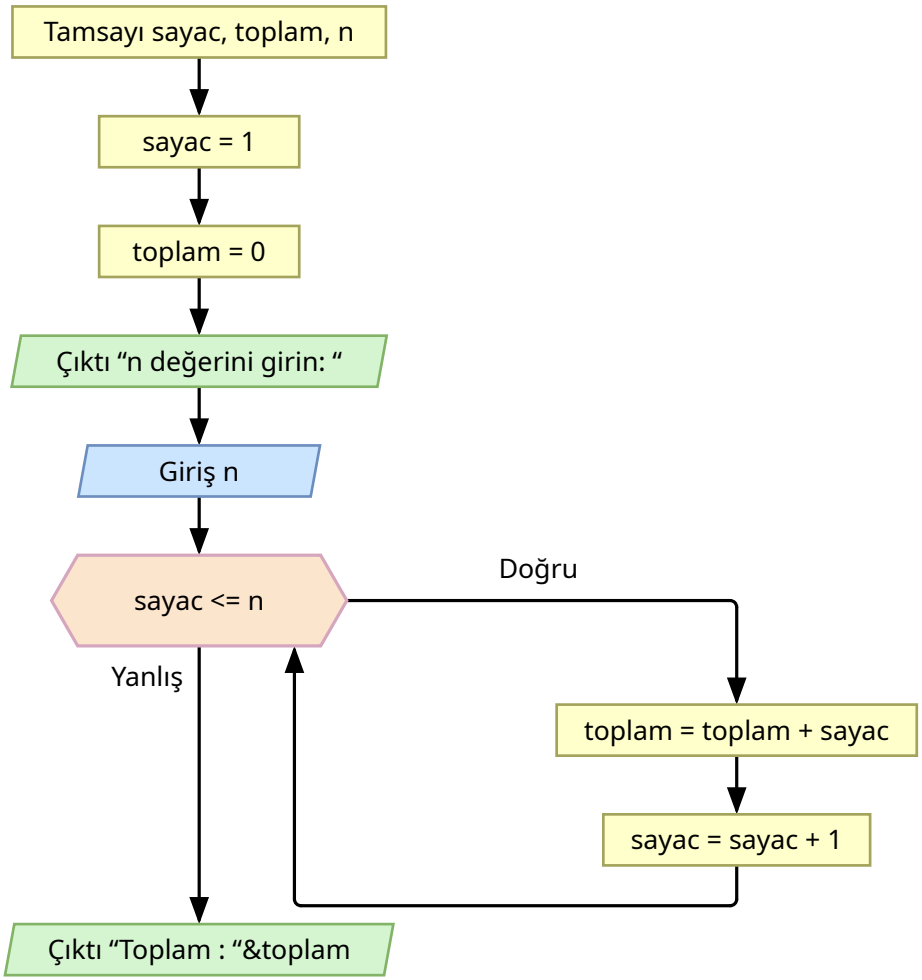
   **Döngüden çık**
6. Adım: **“Toplam = “&toplam** toplam değerini yazdır.

**Elle Çalışma Tablosu**

**Örnek Girdiler:** n = 3 için, **Beklenen Çıktı:** toplam = 6

| Ad | İşlem | sayac | toplam | n | Döngü Koşul (sayac <= n) | Çıktı | Açıklama |
|---|---|---|---|---|---|---|---|
| 1 | Tam sayı sayac, toplam, n | - | - | - | - | - | Kullanılacak değişkenler tanımlanır |
| 2 | sayac = 1, toplam = 0 | 1 | 0 | - | - | - | sayac ve toplam değişkenleri ilk değerlerini alır |
| 3 | n = 3 | 1 | 0 | 3 | 1 <= 3 **evet** | - | koşul doğru sayac = 2, toplam = 0 + 1 = 1 |
| 4 | n = 3 | 2 | 1 | 3 | 2 <= 3 **evet** | - | koşul doğru sayac = 3, toplam =1 + 2 = 3 |
| 5 | n = 3 | 3 | 3 | 3 | 2 <= 3 **evet** | - | koşul doğru sayac = 4, toplam = 3 + 3 = 6 |
| 6 | n = 3 | 4 | 6 | 3 | 4 <= 3 **HAYIR** | “Toplam :” 6 | koşul yanlış sayac = 4, toplam = 6 |

**Akış Şeması (Flowchart)**



### 1.2.2.12. Örnek Çalışma Soruları Döngü Konusu

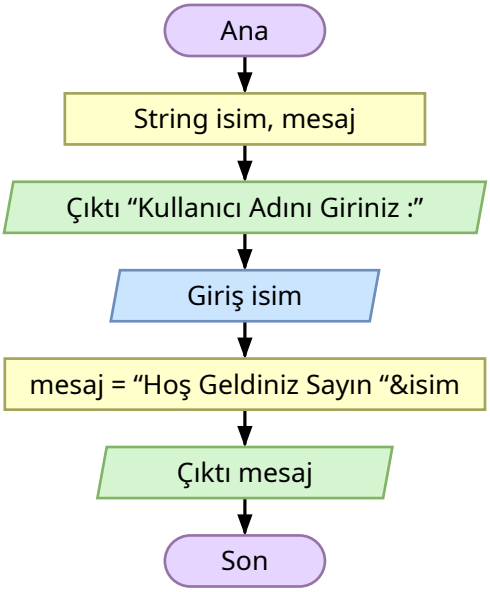
1. 1 ile 100 arasında belirlenen gizli bir sayının (örneğin 33) kullanıcının tahminleriyle bulunmaya çalışıldığı bir tahmin oyununun akış şemasını çiziniz. Döngü kontrolü için boolean tipinde dongu değişkenini kullanınız. Girilen tahmin gizli sayıdan farklı olduğu sürece kullanıcıyı “Sayı Küçük!” veya “Sayı Büyük!” şeklinde yönlendiriniz. Her tahmini sayarak doğru sonuca ulaşıldığında toplam deneme sayısını ekrana yazdırınız.
2. Geri Sayım ve Fırlatma (Geriye Doğru Sayıcı Mantığı): Bir roket fırlatma simülasyonu için 10′dan başlayıp 1′e kadar birer birer geriye doğru sayan ve her sayıyı ekrana yazdıran, döngü bittiğinde ise ekrana “Fırlatma Başarılı!” mesajı veren programın akış şemasını çiziniz.
   İpucu: Döngü sayacını her adımda 1 azaltınız (SAYAC = SAYAC - 1).
3. Çift Sayıların Çarpımı (Faktöriyel / Kümülatif Çarpım Mantığı): Kullanıcıdan pozitif bir çift sayı (N) alarak, 2′den başlayıp N değerine kadar olan (N dahil) tüm çift sayıların çarpımını hesaplayan programın akış şemasını ve sözde kodunu hazırlayınız.
   Örnek Hesap: N = 6 için Carpim = 2 * 4 * 6 = 48
   Öğrenci Notu: Çarpım için başlangıç etkisiz elemanını (CARPIM = 1) ve adım artışını (SAYAC = SAYAC + 2) doğru belirleyiniz.
4. Sınıf Not Ortalaması Hesaplama (Döngü İçinde Girdi Alma ve Ortalama): Kullanıcıdan önce sınıfta kaç öğrenci olduğu bilgisini (ogrenciAdet) alan, ardından döngü yardımıyla bu adet kadar öğrenci notunu (ogrenciNot) teker teker kullanıcıdan isteyip toplayan, döngü sonunda ise sınıfın genel not ortalamasını hesaplayıp ekrana yazdıran programın akış şemasını çiziniz.
   Elle Çalışma: ogrenciAdet = 3 ve girilen notlar 70, 80, 90 olacak şekilde elle çalışma (izleme) tablosunu oluşturunuz.
5. PIN Kodu ile Telefon Kilidi Açma (3 Hak Sınırlı): Telefonun kayıtlı PIN kodu dogruPin = 1234 olarak belirlenmiştir. Kullanıcıya en fazla 3 deneme hakkı tanınacaktır.
   Kullanıcı doğru PIN’i girerse ekrana “Kilit Açıldı” yazıp döngü derhal sonlanmalıdır.
   Kullanıcı 3 hakkında da hatalı giriş yaparsa ekrana “Telefon Kilitlendi!” uyarısı verilmelidir.
   Bu senaryoyu gerçekleştiren programın akış şemasını çiziniz.
6. Sıfır Girilene Kadar Pozitif Sayıları Sayma: Kullanıcı klavyeden 0 (sıfır) girene kadar sürekli sayı girişi kabul eden bir program tasarlayınız.
   Girilen sayılar arasından yalnızca pozitif olanların kaç adet olduğunu saymalıdır (pozitifAdet).
   Kullanıcı 0 girdiğinde döngüden çıkılmalı ve toplamda kaç adet pozitif sayı girildiği ekrana yazdırılmalıdır.

## 1.2.3. Algoritmalarda Metinsel Veriler ve İşlemler

Bilgisayarlar, sayılar dışında isim ve şifre gibi metinsel verileri de işler. Çift tırnak içinde yazılan metinlere **`string`** (**karakter dizisi**) denir. Metinsel veriler doğrudan matematiksel işlemlerde kullanılamaz.

### 1.2.3.1. Örnek Metin Birleştirme (Sayfa 42 Örnek 9)

Kullanıcıdan aldığı isim bilgisiyle ekrana “Hoş Geldiniz Sayın :” + isim şeklinde yazı yazdıran kodun akış şemasını çiziniz?




## 1.2.4. Dijital Depo

Yazılım geliştirme sürecinde kaynak kodların güvenle saklandığı, sürümlerinin takip edildiği alanlara dijital depo (repository) denir. Günümüzde GitHub ve GitLab gibi platformlar; projelerin sürümler halinde yönetilmesini, dağıtılmasını ve ekiplerce ortak geliştirilmesini sağlar. Her deponun vitrini sayılan README.md dosyası ise projenin amacını ve kullanımını özetleyen teknik imzasıdır.

Sizlerin yapacağı yazılım projeleri de ilerleyen derslerimizde GitHub üzerinde yayımlanarak paylaşıma açılacaktır.

**Ahmet Tuna POTUR GitHub Hesabı**

Geliştirdiğim açık kaynaklı yazılımları, eğitim materyallerini ve projelerin sürüm geçmişlerini incelemek için kişisel GitHub profilimi ziyaret edebilirsiniz.

**A.Tuna POTUR GitHub**

Bağlantıya tıklayın
github.com/tunapotur

**Programlamanın Temelleri 9 GitHub Projesi**

Ders notlarımızın kaynak kodlarını barındıran örnek dijital depoyu buradan inceleyebilirsiniz. Projenin amacını öğrenmek için vitrin belgesi olan README.md dosyasını ve geliştirme sürecindeki sürüm adımlarını gözlemleyebilirsiniz.

**GitHub Prog. Temelleri**

Bağlantıya tıklayın
kisa.link/WeLYU

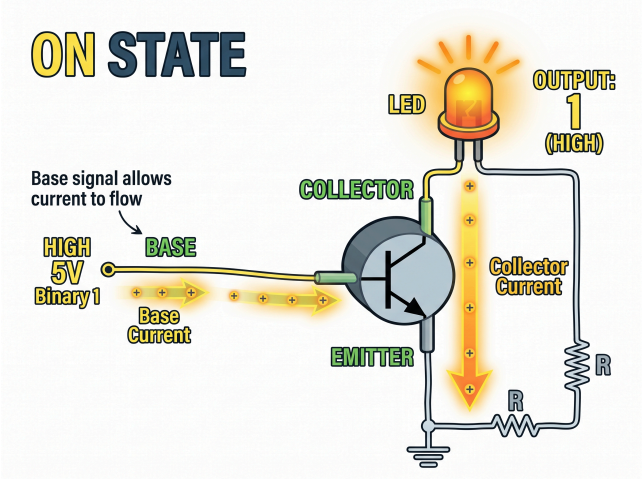
# 2. Temel Programlama Yapıları

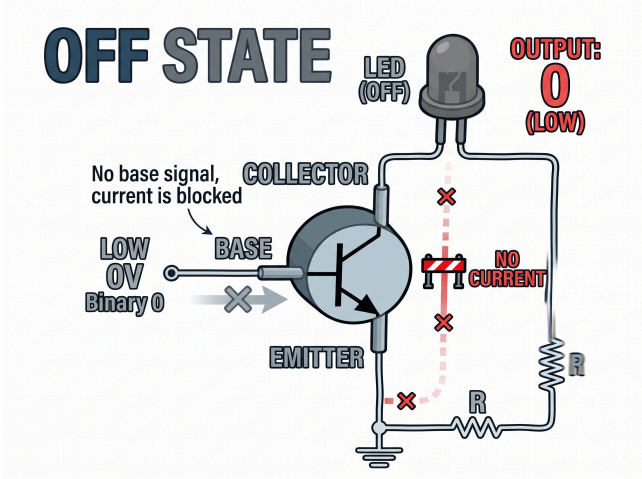
## 2.1. Değişkenler, Sabitler ve Veri Türleri

### 2.1.1. Donanımın Temel Yapı Taşı: Transistör ve İkili Sistem (Binary)

**Transistör** elektrik akımını geçirerek ya da keserek $1$ veya $0$ değerini üreten bilgisayarın en temel donanım birimidir. İşlemci, hafıza gibi bilgisayar donanımlarında milyarlarca transistör bulunur.

**Bir Transistör Elektrik Akını Geçirir veya Keser**




- Elektrik Akımı Var (Açık) : 1 (Bit)
- Elektrik Akımı Yok (Kapalı) : 0 (Bit)

$1$ ve $0$ durumunu oluşturan en küçük bilgi birimine **Bit (Binary Digit)** denir. Bilgisayar dünyasındaki tüm yazılımlar, videolar, oyunlar ve metinler fiziksel temelde sadece $1$ ve $0$'lardan ibarettir.

Yazılım tarihinin en başında, bilgisayarlara bir iş yaptırmak için işlemcinin tek doğal dili olan Makine Dili (**İkilik Sistem / Binary**) kullanılıyordu. Programcılar, komutları `1011 0000` gibi $0$ ve $1$ dizileri halinde delikli kartlara veya anahtarlara işliyordu. Ancak bu yöntem insan gözü için son derece yorucuydu ve tek bir rakamın yanlış yazılması tüm programı bozuyordu.

### 2.1.2. İlk Bilgisayarlar ve 8 Bitlik Komut Girişleri (Altair 8800)

1970'lerin ortalarında kişisel olarak kullanılabilen **Altair 8800** gibi sistemlerin klavyesi veya ekranı yoktu. Ön panelinde sıra sıra fiziksel anahtarlar (**toggle switches**) ve LED lambalar yer alıyordu. Bu bilgisayar, sadece $1$ ve $0$ değerleri kullanılarak donanıma nasıl komut verildiğini ve hesap yaptırıldığını göstermek için somut bir örnektir.

```
+-------------------------------------------------------------+
|                  ALTAIR 8800 ÖN PANELİ                      |
|                                                             |
|   LED'ler (Çıkış):    (O) (X) (O) (O)   (X) (X) (O) (X)     |
|                        0   1   0   0     1   1   0   1      |
|                                                             |
|   Anahtarlar (Giriş): [v] [^] [v] [v]   [^] [^] [v] [^]     |
|                        0   1   0   0     1   1   0   1      |
+-------------------------------------------------------------+
```

- Programcılar her bir $8$ bitlik komutu anahtarları tek tek yukarı ($1$) ve aşağı ($0$) indirerek belleğe elle yüklüyordu.
- İki sayıyı toplamak için bile onlarca $8$ bitlik komutu hatasız girmek gerekiyordu. Bu süreç son derece yavaş, zihinsel olarak yorucu ve hata yapmaya açıktı.

**Tarihi Eser Bilgisayarı Programladım | Altair 8800**

Ömer Bektaş'ın kanalında Altair 8800 hakkında detaylı bir video bulunmakta.

**www.youtube.com/watch?v=1zwTRA0yaYo**

**Altair 8800 Emulatorü**

Altair 8800 cihazını denemek istiyorsanız tarayıcınızda kullanabileceğiniz Altair 8800 Emulator linki.

**www.s2js.com/altair/sim.html**

**Ömer Bektaş'ın Youtube Kanalı**

Ömer Bektaş, bilişim dünyasına yönelik çok faydalı içerikler üreten bir YouTube içerik üreticisidir. Bu kanalı takip etmeniz sizin için çok faydalı olacaktır.

**www.youtube.com/@merbekta**

## 2.1.3. Sayı Sistemleri ve Gruplama: 4 Bit ve Onaltılık (Hexadecimal)

İkilik Sistemde oluşan zorlukları aşmak için **Onaltılık (Hexadecimal)** Sayı Sistemi kullanılmaya başlandı. Dört adet ikilik (binary) basamağın tek bir hexadecimal karaktere (0-9 ve A-F arası) karşılık gelmesi sayesinde, **`1011 0000`**, **`0110 0001`** gibi karmaşık bir makine kodu **`B0`**, **`61`** şeklinde çok daha kısa ve okunabilir bir biçimde ifade edildi. Bu yöntem yazımı kısaltıp hata payını azaltsa da, programcıların hâlâ hangi sayı kodunun hangi işlemi yaptığını ezbere bilmesi gerekiyordu.
$2^4 = 16$ olduğu için 4 bitlik bir binary sayı tam olarak bir hexadecimal basamağı karşılık gelir.

**4-Bit İkilik (Binary) ve Onaltılık (Hexadecimal) Karşılıklar Tablosu**

| Hex | Dec | Binary | Hex | Dec | Binary | Hex | Dec | Binary | Hex | Dec | Binary |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 0000 | **4** | 4 | 0100 | **8** | 8 | 1000 | **C** | 12 | 1100 |
| **1** | 1 | 0001 | **5** | 5 | 0101 | **9** | 9 | 1001 | **D** | 13 | 1101 |
| **2** | 2 | 0010 | **6** | 6 | 0110 | **A** | 10 | 1010 | **E** | 14 | 1110 |
| **3** | 3 | 0011 | **7** | 7 | 0111 | **B** | 11 | 1011 | **F** | 15 | 1111 |

**Hex:** Hexadecimal(16'lık), **Dec:** Decimal(10'luk), **Binary:** Binary(4 Bit),

## 2.1.4. Metin Temsili: 8 Bit = 1 Byte ve ASCII Tablosu

Bir harfi veya sembolü sayılarla temsil etmek için standart referans tablosu **ASCII (American Standard Code for Information Interchange)** geliştirilmiştir. Böylelikle en temel düzeyde gerekli olan karakterlere için **ASCII** tablosundaki karakterin sayısal bir karşılığı oluşturuldu.

- Bilgisayar mimarisinin temelinde $1$ karakter $1$ Byte ($8$ bit) yer kaplar. ASCII tablosunda $256$ karakter bulunmasının sebebi $8$ bit'in yan yana getirilerek $1$ Byte oluşturmasıdır. $1$ Byte aynı zamanda iki hexadecimal karakterle de kolaylıkla ifade edilebilir.

- **Kontrol Karakterleri:** Tablodaki $0$ ile $31$ arasındaki ilk $32$ karakter ve $127$ numaralı **DEL** karakteri ekranda basılabilir bir harf veya sembol değildir. Bunlar cihazlar arası iletişimi ve imleç hareketlerini yöneten **Kontrol Karakterleri (Control / Non-printable Characters)** olarak adlandırılır.

> **ASCII Code - Karakter Kodlama Tablosu**
>
> Tüm standart ve genişletilmiş ASCII karakterleri, onluk (dec), onaltılık (hex) ve ikilik (bin) değerleri ile kontrol karakterlerini incelemek için bu referans tablosunu kullanabilirsiniz.
>
> **www.ascii-code.com**

**Temel ASCII Tablosu (32-127 Karakterin Kodu)**

| Dec | Hex | Binary | Chr | Dec | Hex | Binary | Chr | Dec | Hex | Binary | Chr | Dec | Hex | Binary | Chr |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 32 | 20 | 0010 0000 | *SP* | 56 | 38 | 0011 1000 | 8 | 80 | 50 | 0101 0000 | P | 104 | 68 | 0110 1000 | h |
| 33 | 21 | 0010 0001 | ! | 57 | 39 | 0011 1001 | 9 | 81 | 51 | 0101 0001 | Q | 105 | 69 | 0110 1001 | i |
| 34 | 22 | 0010 0010 | " | 58 | 3A | 0011 1010 | : | 82 | 52 | 0101 0010 | R | 106 | 6A | 0110 1010 | j |
| 35 | 23 | 0010 0011 | # | 59 | 3B | 0011 1011 | ; | 83 | 53 | 0101 0011 | S | 107 | 6B | 0110 1011 | k |
| 36 | 24 | 0010 0100 | $ | 60 | 3C | 0011 1100 | < | 84 | 54 | 0101 0100 | T | 108 | 6C | 0110 1100 | l |
| 37 | 25 | 0010 0101 | % | 61 | 3D | 0011 1101 | = | 85 | 55 | 0101 0101 | U | 109 | 6D | 0110 1101 | m |
| 38 | 26 | 0010 0110 | & | 62 | 3E | 0011 1110 | > | 86 | 56 | 0101 0110 | V | 110 | 6E | 0110 1110 | n |
| 39 | 27 | 0010 0111 | ' | 63 | 3F | 0011 1111 | ? | 87 | 57 | 0101 0111 | W | 111 | 6F | 0110 1111 | o |
| 40 | 28 | 0010 1000 | ( | 64 | 40 | 0100 0000 | @ | 88 | 58 | 0101 1000 | X | 112 | 70 | 0111 0000 | p |
| 41 | 29 | 0010 1001 | ) | 65 | 41 | 0100 0001 | A | 89 | 59 | 0101 1001 | Y | 113 | 71 | 0111 0001 | q |
| 42 | 2A | 0010 1010 | * | 66 | 42 | 0100 0010 | B | 90 | 5A | 0101 1010 | Z | 114 | 72 | 0111 0010 | r |
| 43 | 2B | 0010 1011 | + | 67 | 43 | 0100 0011 | C | 91 | 5B | 0101 1011 | [ | 115 | 73 | 0111 0011 | s |
| 44 | 2C | 0010 1100 | , | 68 | 44 | 0100 0100 | D | 92 | 5C | 0101 1100 | \ | 116 | 74 | 0111 0100 | t |
| 45 | 2D | 0010 1101 | - | 69 | 45 | 0100 0101 | E | 93 | 5D | 0101 1101 | ] | 117 | 75 | 0111 0101 | u |
| 46 | 2E | 0010 1110 | . | 70 | 46 | 0100 0110 | F | 94 | 5E | 0101 1110 | ^ | 118 | 76 | 0111 0110 | v |
| 47 | 2F | 0010 1111 | / | 71 | 47 | 0100 0111 | G | 95 | 5F | 0101 1111 | _ | 119 | 77 | 0111 0111 | w |
| 48 | 30 | 0011 0000 | 0 | 72 | 48 | 0100 1000 | H | 96 | 60 | 0110 0000 | ` | 120 | 78 | 0111 1000 | x |
| 49 | 31 | 0011 0001 | 1 | 73 | 49 | 0100 1001 | I | 97 | 61 | 0110 0001 | a | 121 | 79 | 0111 1001 | y |
| 50 | 32 | 0011 0010 | 2 | 74 | 4A | 0100 1010 | J | 98 | 62 | 0110 0010 | b | 122 | 7A | 0111 1010 | z |
| 51 | 33 | 0011 0011 | 3 | 75 | 4B | 0100 1011 | K | 99 | 63 | 0110 0011 | c | 123 | 7B | 0111 1011 | { |
| 52 | 34 | 0011 0100 | 4 | 76 | 4C | 0100 1100 | L | 100 | 64 | 0110 0100 | d | 124 | 7C | 0111 1100 | \| |
| 53 | 35 | 0011 0101 | 5 | 77 | 4D | 0100 1101 | M | 101 | 65 | 0110 0101 | e | 125 | 7D | 0111 1101 | } |
| 54 | 36 | 0011 0110 | 6 | 78 | 4E | 0100 1110 | N | 102 | 66 | 0110 0110 | f | 126 | 7E | 0111 1110 | ~ |
| 55 | 37 | 0011 0111 | 7 | 79 | 4F | 0100 1111 | O | 103 | 67 | 0110 0111 | g | 127 | 7F | 0111 1111 | *DEL* |

## 2.1.5. Dijital Dünyanın Ölçü Birimi Bit'ten Terabayt'a Veri Büyüklükleri

Bilgisayar dünyasındaki tüm içeriklerin fiziksel temelde sadece $1$ ve $0$'lardan ibaret olduğunu ve $1$ ve $0$ durumun oluşturan en küçük bilgi birimine **Bit (Binary Digit)** dendiğini söylemiştik.

- **Bit (Binary Digit):** Bilgisayardaki en küçük mantıksal veri birimidir. Sadece **0** veya **1** değerini alabilir. Devredeki tek bir transistör lamba anahtarı gibi düşünülebilir: ya kapalıdır (0) ya da açıktır (1).
- **Byte:** 8 adet bitin yan yana gelmesiyle oluşan veri kümesidir.

$$8 \ \text{Bit} = 1 \ \text{Byte}$$

Örneğin **"A"** harfi bellekte **1 Byte** yer kaplar gibi düşünebiliriz.

- Günlük hayatta metrik sistemde birimler onluk sisteme ($10^3$) göre biner biner büyür (1 kg = 1000 g, 1 km = 1000 m). Ancak bilgisayarlar **2'lik sayı sistemine (Binary)** göre çalışır. Bilgisayar mimarisinde bellek adreslemesi 2'nin üsleri şeklinde katlanarak ilerler. Onluk sistemdeki 1000 sayısına ikili tabanda en yakın kuvvet: 1024 sayısıdır.

$$2^{10} = 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 = 1024$$

Bu nedenle bilişim mimarisinde her bir büyüklük basamağı bir öncekinin **1024 katı** ($2^{10}$) kabul edilir.

**Veri Büyüklüğü Basamakları**

| Birim | Kapasite | Kısaltma | Somut Karşılığı |
|---|---|---|---|
| Bit | $0$ veya $1$ | **b** | Tek bir transistör / elektrik sinyali |
| Byte | $8 \ \text{Bit}$ | **B** | Tek bir karakter (harf, rakam, sembol) |
| Kilobyte | $1024 \ \text{Byte}$ | **KB** | Düz metinden oluşan 1 sayfalık belge |
| Megabyte | $1024 \ \text{KB}$ | **MB** | 1 dakikalık MP3 ses kaydı veya telefon fotoğrafı |
| Gigabyte | $1024 \ \text{MB}$ | **GB** | Yaklaşık 1 saatlik HD kalitede film |
| Terabyte | $1024 \ \text{GB}$ | **TB** | Geniş harici yedekleme diski, çok sayıda büyük oyun |
| Petabyte | $1024 \ \text{TB}$ | **PB** | Büyük veri merkezleri veya bulut sunucu havuzları |
| Exabyte | $1024 \ \text{PB}$ | **EB** | Küresel internet trafiğinin aylık ölçekteki hacmi |

> **Donanım Üreticisi vs. İşletim Sistemi Farkı:** Piyasadan satın alınan **1TB** kapasiteli bir SSD veya harici disk bilgisayara takıldığında, depolama alanı genellikle yaklaşık **931 GB** görünür. Bu durum donanım hatası değildir:
>
> Donanım Üreticileri **Onluk** sayı birimini kullanarak $1 \ \text{KB} = 10^3 \ \text{Byte}$ şeklinde hesap yapar:
>
> $$1 \ \text{TB} = 1.000.000.000.000 \ \text{Byte}$$
>
> Fakat işletim sistemleri ikilik sayı sistemine göre ölçüm yaptığı için altta ki bölme işleminin sonucu satın aldığınız diskinizin gerçek boyutu **1TB** altında olacaktır.
>
> $$\frac{1.000.000.000.000}{1024 \times 1024 \times 1024} \approx 931.32 \ \text{GB}$$
>
> Sizce bu disk üreticilerinin yaptığı bir hile midir?

## 2.1.6. Makine Kodundan Assembly Diline

Altair 8800 gibi erken dönem bilgisayarlarda, işlemciye komut vermek için sürekli **`1` ve `0`** dizilimlerinden oluşan makine kodlarını kullanmak oldukça zahmetliydi ve hatalara açıktı. Makine kodlarını daha kısa ve okunabilir biçimde ifade etmek için **Onaltılık (Hexadecimal) sayı sistemi** kullanılmaya başlandı. Ancak hexadecimal gösterim, makine kodunu daha kolay yazmayı sağlasa da komutların anlamını doğrudan açıklamıyordu.

Bu nedenle makine kodundaki komutları daha kolay hatırlayabilmek ve yazabilmek için İngilizce kelimelerin kısaltmalarından oluşan sembolik isimler (**Mnemonic**) kullanılmaya başlandı. Bu sembolik komutları kullanan dile **Assembly dili** adı verildi. Böylece programcılar, yalnızca `1` ve `0` dizilimleriyle uğraşmak yerine, daha anlaşılır ve hatırlanması kolay komutlarla program yazabilir hâle geldi.

**Hexadecimal Yerine Assembly Kullanımı Örneği**

| Hexadecimal | Yerine | Assembly | Açıklama |
|---|---|---|---|
| `B0 61` | ⟶ | `MOV AL, 61H` | AL yazmacına **`61H`** verisi taşındı. |
| `04 05` | ⟶ | `ADD AL, 05H` | AL yazmacındaki değere **`05H`** eklendi. |
| `2C 02` | ⟶ | `SUB AL, 02H` | AL yazmacındaki değerden **`02H`** çıkarıldı. |

Bu sembolik komutlarla yazılan programlar, **Assembler (Çevirici)** adı verilen bir program aracılığıyla işlemcinin anlayabileceği makine koduna dönüştürüldü.

**x86 Assembly: İki Sayıyı Toplama Örneği**

```
section .text
global _start

_start:
    mov eax, 5          ; ilk sayı
    mov ebx, 7          ; ikinci sayı
    add eax, ebx        ; eax = eax + ebx  (5 + 7 = 12)

    mov ebx, eax        ; sonucu ebx'e al (exit kodu ebx'ten okunur)
    mov eax, 1          ; sys_exit sistem çağrısı numarası
    int 0x80            ; çekirdeği çağır
```

**Dillerin Atası | Assembly**

Ömer Bektaş'ın kanalında Assembly dili hakkında detaylı bir video bulunmakta.

**www.youtube.com/watch?v=yWXuG3RLkTE**

## 2.1.7. Programlama Dilleri

Her ne kadar $1$, $0$ veya hexadecimal sayılar ile kodlama yapmaya göre Assembly dili büyük bir kolaylık sağlasa da projeler büyüdükçe kodları yazmak zorlaşıyordu. İşte bu sorunu çözmek için **Fortran, COBOL ve C** gibi programlama dilleri ortaya çıktı. Artık iki sayıyı toplamak için işlemci yazmaçlarıyla uğraşmak yerine doğrudan **`sonuc = a + b`** yazmak yeterli hale geldi. Programlama dilleri ile yazılan kodlar, **Derleyici (Compiler)** veya **Yorumlayıcı (Interpreter)** adı verilen araçlarla arka planda tekrar makine koduna çevrilerek işlemci üzerinde çalıştırılmaktadır.

**Derleyici (Compiler):** Programcının yazdığı kodun tamamını önce baştan sona okur, yazım hatalarını kontrol eder ve tüm kodu bir bütün olarak makine diline çevirir. Kodun yazımında (söz diziminde) bir hata varsa derleme işlemi tamamlanmaz.

**Yorumlayıcı (Interpreter):** Kodu bir bütün olarak değil satır satır okuyarak anında çalıştırır. Bir satır çevrilip çalıştırıldıktan sonra sıradaki satıra geçilir. Hata tespiti açısından yorumlayıcı, hatanın bulunduğu satıra gelene kadar programı çalıştırır ve o noktada durur. Bu yaklaşım tıpkı bir tercümanın, konuşmacının cümlelerini anlık olarak çevirmesine benzer. **Python ve Ruby**, bu yaklaşımı kullanan dillere örnek gösterilebilir.

| Evrim Aşaması | Temsil Biçimi | Örnek Kod | Temel Avantajı | Temel Zorluğu |
|---|---|---|---|---|
| **1. Makine Dili (Binary)** | `0` ve `1` dizileri | `1011 0000`<br>`0110 0001` | İşlemci hiçbir çeviriye ihtiyaç duymadan doğrudan çalıştırır. | İnsan tarafından okunması ve yazılması çok zordur. |
| **2. Hexadecimal (16'lık Sistem)** | `0-9` ve `A-F` karakterleri | `B0 61` | Uzun 0 ve 1 dizilerini 4'lü gruplar halinde kısaltarak yazım kolaylığı sağlar. | Komutların sayısal karşılıklarını akılda tutmak gerekiyordu. |
| **3. Assembly Dili (Sembolik Dil)** | Sembolik kısaltmalar *(Mnemonics)* | `MOV AL, 61h` | Komutlar (`MOV`, `ADD`, `SUB`) anlamlı kelimelerle ifade edilir. | Her işlemci mimarisi için farklıdır; büyük projelerde kod çok uzar. |
| **4. Yüksek Seviyeli Diller** | Günlük dil ve matematiksel ifadeler | `int a = 97;` | Öğrenmesi kolaydır, donanımdan bağımsızdır ve hızlı yazılım geliştirmeyi sağlar. | İşlemcide çalışabilmesi için derleyici veya yorumlayıcı ile çevrilmesi gerekir. |

# Bilgisayarın Dili

Transistörden yüksek seviyeli programlama dillerine: 1 ve 0'ın yolculuğu

## Programlamanın Evrimi

**1 Makine Dili**

`1011 0000`

✓ İşlemci çeviri olmadan çalıştırır
✗ Okuması ve yazması çok zor

**2 Hexadecimal**

`B0 61`

✓ 0/1 dizilerini 4'lü gruplarla kısaltır
✗ Komut kodları ezberlenmeli

**3 Assembly**

`MOV AL, 61H`

✓ Anlamlı kısaltmalar (MOV, ADD)
✗ Her işlemciye özel, kod uzar

**4 Yüksek Seviyeli**

`sonuc = a + b`

✓ Kolay, donanımdan bağımsız, hızlı
✗ Derleyici/yorumlayıcı gerekir

### 1 · Transistör: Akımı Geçirir veya Keser

Bilgisayarın en temel donanım birimi; işlemci ve hafızada **milyarlarca** transistör bulunur.

**AÇIK · Çıkış 1**
Base: HIGH (5V)
Akım geçer, LED yanar

**KAPALI · Çıkış 0**
Base: LOW (0V)
Akım kesilir, LED söner

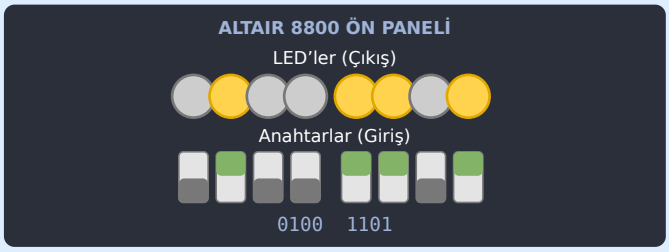
### 2 · Bit (Binary Digit)

1 ve 0 durumunu oluşturan **en küçük bilgi birimi.** Yazılımlar, videolar, oyunlar ve metinler fiziksel olarak yalnızca 1 ve 0'lardan oluşur.

1 0 1 1

### 3 · Altair 8800 (1970'lerin ortası)

Klavye ve ekran yoktu; ön panelde **toggle switch** ve LED'ler vardı. Her 8 bitlik komut anahtarlarla tek tek belleğe yükleniyordu: yukarı = 1, aşağı = 0.




İki sayıyı toplamak için bile onlarca komut hatasız girilmeliydi: yavaş, yorucu, hataya açık.

### 4 · Hexadecimal: 4 Bit = 1 Basamak

$2^4 = 16$ olduğundan 4 bit, tek bir hex karakterine (0–9, A–F) karşılık gelir.

| | | | |
|---|---|---|---|
| **0** (0) 0000 | **1** (1) 0001 | **2** (2) 0010 | **3** (3) 0011 |
| **4** (4) 0100 | **5** (5) 0101 | **6** (6) 0110 | **7** (7) 0111 |
| **8** (8) 1000 | **9** (9) 1001 | **A** (10) 1010 | **B** (11) 1011 |
| **C** (12) 1100 | **D** (13) 1101 | **E** (14) 1110 | **F** (15) 1111 |

`1011 0000` → **B0** `0110 0001` → **61**

### 5 · Metin Temsili: 8 Bit = 1 Byte ve ASCII

Harf ve sembolleri sayılarla temsil etmek için **ASCII** standardı geliştirildi. 1 karakter = **1 Byte** (8 bit) yer kaplar ve iki hex karakterle yazılır.

0 1 0 0 0 0 0 1

1 Byte = hex **41** = onluk **65** = '**A**'

| Chr | Dec | Hex | Binary |
|---|---|---|---|
| SP | 32 | 20 | 0010 0000 |
| 0 | 48 | 30 | 0011 0000 |
| A | 65 | 41 | 0100 0001 |
| a | 97 | 61 | 0110 0001 |
| DEL | 127 | 7F | 0111 1111 |

**0–31 ve 127** Kontrol karakterleri (basılamaz) | **32–47** Boşluk ve semboller | **48–57** Rakamlar 0–9 | **65–90** Büyük harfler A–Z | **97–122** Küçük harfler a–z

### 6 · Makine Kodundan Assembly'ye

Komutlar, İngilizce kelime kısaltmalarından oluşan sembolik isimlerle (**mnemonic**) yazıldı. **Assembler** bu kodu makine koduna çevirir.

| Hex | Assembly | Açıklama |
|---|---|---|
| B0 61 | MOV AL, 61H | AL'ye 61H taşındı |
| 04 05 | ADD AL, 05H | AL'ye 05H eklendi |
| 2C 02 | SUB AL, 02H | AL'den 02H çıkarıldı |

**x86 · 5 + 7**

```
mov eax, 5 ; ilk sayı
mov ebx, 7 ; ikinci sayı
add eax, ebx ; 12
Yüksek seviyede: sonuc = a + b
```

### 7 · Programlama Dilleri: Derleyici ve Yorumlayıcı

Fortran, COBOL ve C gibi diller çıktı; kod arka planda yine makine koduna çevrilir.

**Derleyici (Compiler)**

Kodun **tamamını** baştan sona okur, hataları denetler, bütünü makine diline çevirir. Söz diziminde hata varsa derleme tamamlanmaz.

**Yorumlayıcı (Interpreter)**

Kodu **satır satır** okuyup anında çalıştırır; hatalı satıra gelince durur. Tıpkı anlık çeviri yapan bir tercüman gibi (Python, Ruby).

## 2.1.8. Değişkenler ve Temel Veri Türleri

### 2.1.8.1. Değişken

Değişken; programın çalışması sırasında verilerin bellekte (RAM) geçici olarak tutulduğu alanlara verilen isimdir. Kullanıcıdan veya harici bir kaynaktan alınan bilgiler bu değişkenlerde saklanır; kod çalıştıkça hesaplamalarda kullanılır ve süreç içinde yeni değerler alabilir.
Değişkenler yardımıyla bellekte (RAM) saklanan verilere kolayca erişilir; kod çalıştırıldığında bu değişken isimleri çağrılarak temsil ettikleri veriler üzerinde işlemler gerçekleştirilir.

**Örnek Python Kodu**

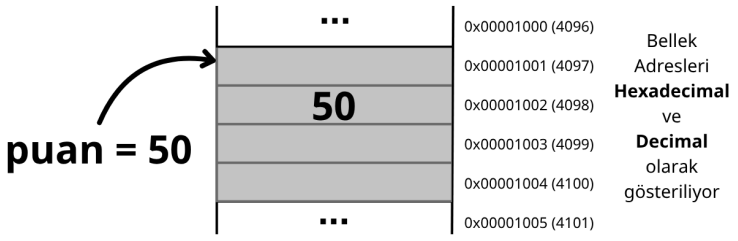
```
puan = int(input("Puan Giriniz: "))
print("Puanı:", puan)
puan = puan + 25
print("Güncellenmiş Puan:", puan)
```

**Python Ekran Çıktısı**

```
Puan Giriniz: 50
Puanı: 50
Güncellenmiş Puan: 75
```

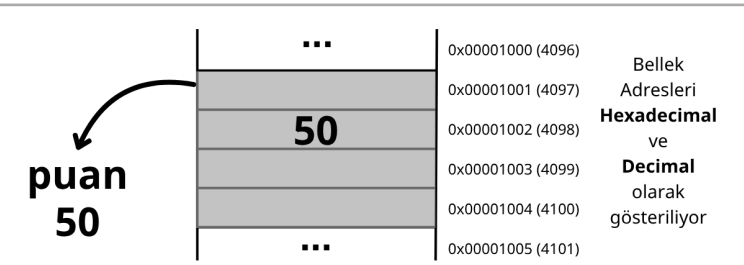
**1. Değer Atama**
Giriş yapılan **50** değeri, RAM'de **puan** ismiyle bir adrese kaydedilir.

```
Puan Giriniz: 50
```

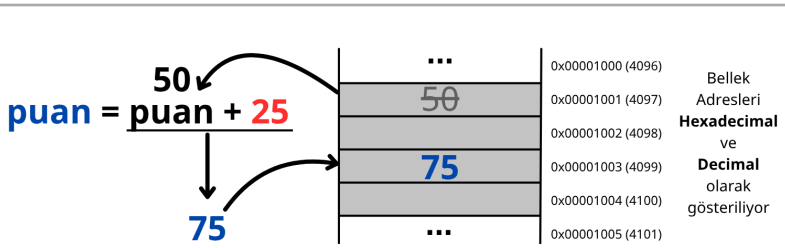



**2. Hafızadan Çağırma**
**puan** değişkeni RAM'deki adresinden çağrılarak **50** değeri ekrana yazdırılır.

```
Puanı: 50
```

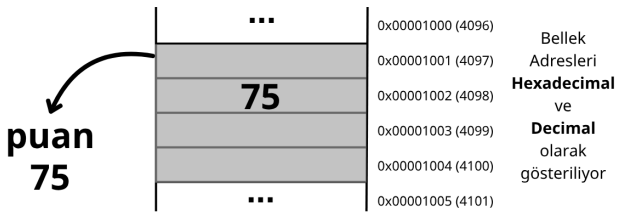



**3. Değer Güncelleme**
Mevcut **50** değeri hafızadan alınır, **25** eklenir ve elde edilen yeni **75** değeri tekrar **puan** değişkenine atanır.




**4. Güncel Hali Çağırma**
Güncellenen **puan** değişkeni hafızadan çağrılarak ekrana yazdırılır.

```
Güncellenmiş Puan: 75
```



### 2.1.8.2. Temel Veri Türleri

Bilgisayar belleğine kaydedilecek verilerin nitelikleri, boyutları ve üzerlerinde hangi matematiksel veya mantıksal işlemlerin yapılabileceği veri türü (data type) kavramı ile belirlenir. Python dilinde değişkenlerin veri türleri, türetildikleri anda dinamik olarak yorumlayıcı tarafından atanır.

**Tam Sayı [Integer (int)]:** Kesirli kısmı bulunmayan; pozitif, negatif veya sıfır değerini alabilen sayısal veri türüdür (örneğin ogrenci_sayisi = 34, hava_sicakligi = –5).

**Ondalıklı Sayı [Floating Point (float)]:** Kesirli veya ondalıklı kısımları bulunan, bilimsel hesaplamalarda ve hassas ölçümlerde kullanılan sayısal veri türüdür. Python dilinde ondalık kısımlar virgül (,) değil uluslararası standart gereği nokta (.) karakteriyle (örneğin urun_fiyati = 89.90, pi_sayisi = 3.14159) ayrılır.

**Metinsel Veri [String (str)]:** Tek tırnak '...' veya çift tırnak "..." işaretleri arasında tanımlanan; alfabetik karakterler, rakamlar ve özel sembollerden oluşan karakter dizileridir. Rakamlardan oluşsalar dahi string türündeki verilerle doğrudan aritmetik işlem yapılamaz. Örneğin kullanici_adi = "gok turk_9", okul_numarasi = "1071"

**Mantıksal Veri [Boolean (bool)]:** Sadece iki farklı durumun kontrolünde kullanılan, mantıksal karar süreçlerinin temelini oluşturan veri türüdür. Alabileceği değerler yalnızca Doğru veya Yanlış ifadeleridir. Örneğin sistem_acik_mi = True, bakiye_yeterli_mi = False

### 2.1.8.3. Değişken Tanımlama Kuralları

Yazılan kaynak kodların Python yorumlayıcısı tarafından doğru şekilde işlenebilmesi için değişken adları belirlenirken Python'un söz dizimi (**syntax**) kurallarına uyulması zorunludur.

**Karakter Kısıtlaması:** Değişken isimleri sadece alfabetik karakterler (**A-Z, a-z**), rakamlar (**0-9**) ve alt çizgi (_) karakterini barındırabilir. Bunların dışındaki hiçbir özel sembol (nokta, virgül, soru işareti, tire, boşluk vb.) değişken isminde yer alamaz.

**Başlangıç Kriteri:** Bir değişken ismi kesinlikle bir rakam ile başlayamaz. İsimler ya alfabetik bir karakterle ya da alt çizgi (_) işaretiyle başlamak zorundadır. Örneğin **3etap_skor** hatalı bir söz dizimi iken, **etap_skor_3** veya **_etap3** kurallara uygundur.

**Büyük-Küçük Harf Duyarlılığı (Case-Sensitivity):** Python dili, büyük ve küçük harfleri farklı karakterler olarak değerlendirir. Bu nedenle aynı harflerden oluşsa bile yazımında büyük-küçük harf farkı bulunan kelimeler, bellekte tutulan verilere erişmek için kullanılan farklı değişken adları olarak kabul edilir. Örneğin **asistan_adi**, **Asistan_Adi** ve **ASISTAN_ADI** ifadeleri Python yorumlayıcısı tarafından birbirinden bağımsız üç farklı değişken adı olarak değerlendirilir.

**Resmî Dil Anahtarları (Reserved Words):** Python dilinin kendi iç mekanizmasını yönetmek ve komutları işletmek adına kendine ayırdığı bazı özel kelimeler (keywords) değişken adı olarak kullanılamaz. Örneğin **if**, **else**, **True**, **False**, **while**, **for** gibi kelimeler Python için özel bir anlam ifade ettiğinden doğrudan değişken ismi yapılamaz.

**Türkçe Karakter Kullanımı:** Kodlama dillerinin ve yazılım araçlarının küresel standardı gereği, değişken isimlendirirken **Türkçe'ye özgü harfleri kullanmaktan kesinlikle kaçınınız!**.

**Türçe Karakterlerin İngilizce Karşılıkları**

| Türkçe Karakter | Doğru Kullanım | İngilizce Karakte |
|---|---|---|
| ç / Ç | → | **c / C** |
| ğ / Ğ | → | **g / G** |
| ı / İ | → | **i / I** |
| ö / Ö | → | **o / O** |
| ş / Ş | → | **s / S** |
| ü / Ü | → | **u / U** |

**Değişken Kullanım Örnekleri**

| Türkçe Karakter | Doğru Kullanım | İngilizce Karakte |
|---|---|---|
| doğumTarihi | → | **dogumTarihi** |
| ürünFiyatı | → | **urunFiyati** |
| şubeKodu | → | **subeKodu** |
| kayıtDurumu | → | **kayitDurumu** |
| şifresi | → | **sifresi** |
| öğrenci_yaşı | → | **ogrenci_yasi** |
| ücret_miktarı | → | **ucret_miktari** |
| ilk_değer | → | **ilk_deger** |

### 2.1.8.4. Değişken Adlandırma Standartları (Notasyonlar)

**Yılan Gösterimi (snake_case)**

Tüm harflerin küçük yazıldığı ve kelimelerin birbirine alt çizgi (_) karakteriyle bağlandığı yöntemdir. Python literatüründe değişken ve fonksiyon isimleri için bu standardın kullanılması (örneğin kullanici_giris_tarihi, asistan_bellek_kapasitesi vb.) kurumsal olarak tavsiye edilir.

**Camel Case (Deve Notasyonu) Kuralı**

Değişken adları tanımlanırken kelimeler arasında boşluk bırakılamaz. Birden fazla kelimeden oluşan değişkenleri okunaklı kılmak için dünya çapında standart olan **camelCase** yöntemi kullanılır.
**Kural:** İlk kelimenin tüm harfleri küçük başlar. Sonraki her kelimenin ilk harfi büyük yazılır (kelimelerin ortasındaki hörgüç görünümü gibi).

| Yanlış Biçim | snake_case | camelCase |
|---|---|---|
| personelsayisi | **personel_sayisi** | **personelSayisi** |
| dogumyil | **dogum_yil** | **dogumYil** |
| ogrencinotortalamasi | **ogrenci_not_ortalamasi** | **ogrenciNotOrtalamasi** |

### 2.1.8.5. Sabitler

Bir yazılım mimarisinde programın çalışma süresince değiştirilmemesi amaçlanan ve aynı değeri koruması beklenen verilere sabit (constant) adı verilir. Sabit olarak kullanılması amaçlanan isimler, tüm harfleri büyük olacak ve kelimeler alt çizgiyle (_) ayrılacak biçimde yazılır.

```
# Doğru Sabit Kullanım Standardı (BÜYÜK HARF)
PI_SAYISI = 3.14159
MAKSIMUM_DENEME_SAYISI = 3
SISTEM_PORT_NUMARASI = 443
VARSAYILAN_DIL = "TR"
KARSILAMA_MESAJI = " Sisteme Hoş Geldiniz!"
GELISTIRICI_MODU_ACIK = True
```

### 2.1.8.6. Açıklama Satırı (Comment line)

Yazılım projelerinde, derleyicinin görmezden geldiği ancak yazılımcıların kendileri için veya diğer yazılımcılar için bıraktığı notlara açıklama sataı (comment line) denir. Python'da iki tür açıklama satırı uygulaması vardır.

1. # ile başlayan tek satırlık açıklama satırı
2. """..."""" Çok satırlı açıklama satırları üç adet tırnak ile açılan ve yine üç adet tırnak ile kapanan alan içine yazılan açıklama ifadeleri ile yapılır.

```python
# Bu tam satırlık açıklama
x = 10  # satır içi (inline) açıklama

# Bu fonksiyon iki sayıyı toplar,
# ardından sonucu ekrana yazdırır
# ve geriye döndürür.
def topla(a, b):
    return a + b
```

```python
"""
Bu teknik olarak bir string değişmezidir,
ancak bir değişkene atanmadığı için
yorum bloğu gibi davranır.
"""
print("Kod çalışıyor")
```

## 2.1.9. Python Dili

**Python**; yüksek seviyeli, yorumlanan, nesne yönelimli ve geniş bir kütüphane ekosistemine sahip bir programlama dilidir. 1991 yılında Guido van Rossum tarafından geliştirilen bu dil, adını van Rossum'un beğeniyle takip ettiği İngiliz komedi grubu Monty Python'ın Monty Python's Flying Circus adlı televizyon programından almıştır.

Python, sade söz dizimi ve yüksek okunabilirliği sayesinde web geliştirmeden veri analizine, robotik kontrolden yapay zekâ sistemlerine kadar çok geniş bir alanda tercih edilir. Sunduğu bu esneklik ve zengin kütüphane desteği nedeniyle Python; küresel yazılım dünyasında olduğu gibi Türkiye'deki teknoloji girişimlerinde, akıllı otomasyon sistemlerinde ve Ar-Ge projelerinde de yaygın olarak kullanılmaktadır.

Günümüzde bilgisayar donanımları, programlama dillerinin ilk ortaya çıktığı dönemlere kıyasla inanılmaz bir kapasiteye ulaşmıştır. Bu sayede Python gibi dillerin getirdiği çalışma zamanı maliyeti modern donanımlarla büyük ölçüde tolere edilmekte; çoğu uygulama için fazlasıyla yeterli ve pratik bir performans sunmaktadır.

**İki Sayının Toplamını Alan Kod Örneği**

**C Dili**

```c
#include <stdio.h>

int main() {
    int sayi1, sayi2, toplam;

    printf("Birinci sayi: ");
    scanf("%d", &sayi1);

    printf("Ikinci sayi: ");
    scanf("%d", &sayi2);

    toplam = sayi1 + sayi2;
    printf("Toplam: %d\n", toplam);

    return 0;
}
```

**Python Dili**

```python
sayi1 = int(input("Birinci sayı: "))
sayi2 = int(input("İkinci sayı: "))

toplam = sayi1 + sayi2
print(f"Toplam: {toplam}")
```